CURS 1

SISTEME GRAFICE : CLASIFICARE

- de **sinteza** a imaginilor
    - aplicatii in industria de jocuri/film
    - domeniu : Computer Graphics(**CG**)
- de **prelucrare/ analiza** a imaginilor / sisteme cu viziune
    - aplicatii in analiza imaginilor medicale, recunoastere obiecte/fete/caractere de text
    - domeniu: Computer Vision(**CV**)
- de **reconstructie si vizualizare** a unor obiecte / fenomene reale pornind de la date volumetrice
    - aplicatii in medicina (vizualizare 3d organe, dinamicii moleculare, date meteo)
    - domeniu : Volume Graphics (combina CG SI CV)

MODELUL 2D/3D AL IMAGINII

- alcatuit din:
    - **primitive grafice 2d,3d** (linii, poligoane) sau obiecte complexe - alcatuiesc **scena** redata intr-un cadru de imagine
    - proprietati de suprafete - **culoare, textura, proprietati de reflexie/refractie lumina**
    - **transformari** pt obiecte in scopul compunerii, vizualizarii scenei si animarii obiectelor
    - **surse de lumina**
    - **POV camera / directia privirii**
- orice modificare a modelului trebuie sa aibe ca efect modificarea in timp real a imaginii afisate

OPERATII DE SINTEZA A IMAGINILOR GRAFICE

- transformari pt modelarea scenei
- transformari 2d -> 3d
- eliminare obiecte care nu se vad in scena / eliminare fete ale obiectelor care nu se vad din cauza pov-ului
- decupare primitive la marginile volumului vizual
- rasterizare = descompunerea primitivelor grafice in fragmente care se afiseaza in pixeli
- calcul culori
- operatii la nivel de pixel

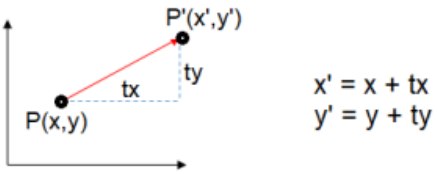
CURS 2 - TRANSFORMARI GEOMETRICE 2D

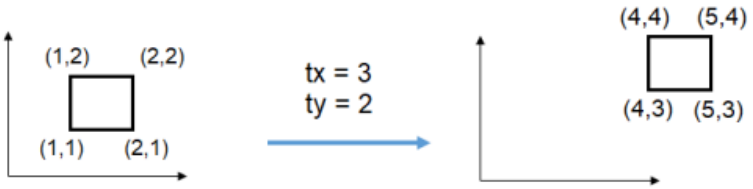
- reprezentare obiecte 2d/3d
    - **date geometrice : coordonate varfuri raportate la oxy sau oxyz**
    - **atribute topologice** - definesc modul in care sunt conectate varfurile
    - **atribute de aspect** - culoarea, tipul de interior pt suprafete 2D, atribute de
- ! transformarile geometrice se aplica varfurilor (coord) si nu afecteaza atributele

- transformarile geometrice sunt fundamentale in sinteza imaginilor
- folosite pentru
    - redarea desenelor la diferite marimi
    - compunerea desenelor/ scenelor 3d
    - realizarea animatiei
    - transformarea obiectelor dintr-un spatiu de coordonate logic (in care sunt definite) in spatiul fizic de afisare

TRANSFORMARI GEOMETRICE 2D

- pentru obiecte 2D (in sistem XOY)
- TRANSFORMARI ELEMENTARE:
    - **translatia**
    - **scalarea fata de origine**
    - **rotirea fata de origine**
    - **forfecarea fata de origine**
    - **oglindire fata de axele principale si fata de origine**
- ! orice alta transformare se obtine prin compunerea a 2 sau mai multe transformari elementare
    - ex. scalare/ rotire fata de un punct oarecare din plan, scalare compusa cu translatie, etc.
- **TRANSLATIA**
    - definita printr-un vector T[tx, ty]
    - **x’ = x + tx**
    - **y’ = y + ty**





- **SCALAREA FATA DE ORIGINE**
    - definita prin 2 numere reale, de regula pozitive
    - sx - scalarea pe OX
    - sy - scalarea pe OY

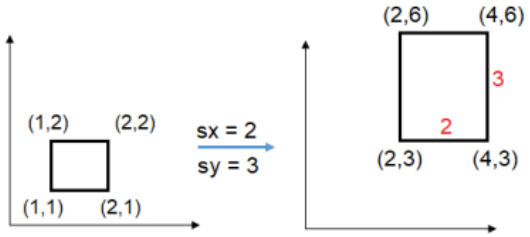
    - **x’ = x * sx**
    - **y’ = y * sy**
    - efecte:
        - daca sx / sy sau sx si sy > 1 -> se mareste

- daca sx / sy sau sx si sy < 1 -> se micsoreaza
- sx = sy scalare unifiorma

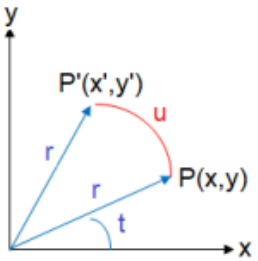
Reprezentarea matriceală

$$[x'\ \ y'] = [x\ \ y] * \begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix} * \begin{bmatrix} x \\ y \end{bmatrix}$$



- **ROTATIA FATA DE ORIGINE**



x = r*cos(t)
y = r*sin(t)

Relația dintre coordonatele carteziene și coordonatele polare ale unui punct P(x,y)

x' = r*cos(t+u) = r*(cos(t)*cos(u) – sin(t)*sin(u)) = x*cos(u) – y*sin(u)
y' = r*sin(t+u) = r*(cos(t)*sin(u) + sin(t)*cos(u)) = x*sin(u) – y*cos(u)

Rotatia fata de origine

x' = x*cos(u) - y*sin(u)
y' = x*sin(u) + y*cos(u)

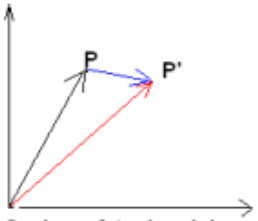
$$[x'\ \ y'] = [x\ \ y] * \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(u) & -\sin(u) \\ \sin(u) & \cos(u) \end{bmatrix} * \begin{bmatrix} x \\ y \end{bmatrix}$$

- **SCALAREA FATA DE UN PUNCT OARECARE DIN PLAN**
  - punctul fix al transformarii este un punct oarecare F(xf, yf) - coordonatele sale nu se modifica prin aplicarea transformarii
  - P(x, y) este punctul transformat
  - scalarea se aplica vectorului FP[x-xf, y-yf]

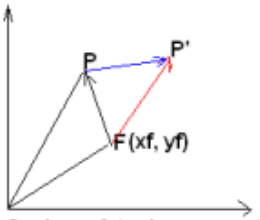
x' – xf = sx*(x – xf)
y' – yf = sy*(y – yf)

Rezultă:
x' = x*sx + xf –xf*sx
y' = y*sy + yf –yf*sy



Scalarea fata de origine

x' - 0 = sx (x - 0)
y' - 0 = sy (y - 0)



Scalarea fata de un punct F din plan

x' - xf = sx (x - xf)
y' - yf = sy (y - yf)

Nu poate fi exprimata în coordonate carteziene printr-o matrice de 2x2!

- **ROTATIA FATA DE UN PUNCT OARECARE DIN PLAN**
  - punctul fix al transformarii ete F(xf, yf)
  - P(x,y) este punctul transformat
  - rotatia se aplica vectorului FP[x-xf, y-yf], in jurul lui F

$x' - xf = (x-xf)^* \cos(u) - (y - yf) * \sin(u)$

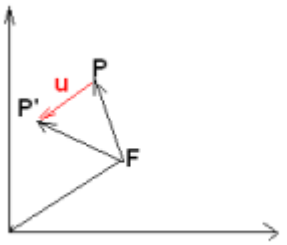
$y' - yf = (x-xf)^* \sin(u) + (y - yf) * \cos(u)$

Rezultă:

$x' = x^*\cos(u) - y^*\sin(u) + xf - xf^*\cos(u) + yf^*\sin(u)$

$y' = x^*\sin(u) + y^*\cos(u) + yf - xf^*\sin(u) - yf^*\cos(u)$

Nu poate fi exprimată în coordonate carteziene printr-o matrice de 2x2!



Rotatia fata de un punct F din plan

- **COMPUNEREA TRANSFORMARILOR GEOMETRICE 2D**
  - matricea unei transformari compuse se obtine prin inmultirea matricilor transformarilor elementare

RS # SR

$$RS = R^*S = \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix} \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

$$SR = S^*R = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix}$$

  - ! **translatia nu poate fi reprezentata in coordonate carteziene printr-o matrice, aceasta impune reprezentarea transformarilor in <u>coordonate omogene</u>**

- **COORDONATE OMOGENE**
  - un punct in plan P(x,y) se reprezinta in coordonate omogene printr-un vector [xw, yw, w]
  - **xw = x * w**
  - **yw = y * w**
  - **w - orice numar real -** pentru a usura transf cartezian->omogen si invers folosim w = 1

Translația

$$[x'\,y'\,1] = [x\,y\,1] \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix} \quad \text{sau} \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Scalarea față de origine

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Rotația fața de origine

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} \cos(u) & \sin(u) & 0 \\ -\sin(u) & \cos(u) & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(u) & -\sin(u) & 0 \\ \sin(u) & \cos(u) & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- **TRANSFORMARILE INVERSE ALE TRANSFORMARILOR ELEMENTARE**

  T(tx, ty): matricea translației, în coordonate omogene

  S(0, 0, sx, sy): matricea scalării față de origine, în coordonate omogene

  R(0,0,u): matricea rotației față de origine, în coordonate omogene

  Transformările inverse sunt:

  $T(tx, ty)^{-1} = T(-tx, -ty)$

  $S(0, 0, sx, sy)^{-1} = S(0, 0, 1/sx, 1/sy)$

  $R(0,0, u)^{-1} = R(0, 0, -u)$

- **TRANSFORMARI GEOMETRICE 2D COMPUSE**
  - expresiile matematice ale **scalarii si rotatiei fata de un punct oarecare din plan** se pot obtine prin compunerea urmatoarelor transformari
    - **translatia** prin care pct fix al transformarii ajunge in origine **T(-xf, -yf)**
    - **scalarea / rotatia** fata de origine : **S(0,0,sx,sy)**
    - **translatia inversa T(xf, yf)** a primei translatii

Scalarea față de punctul F(xf,yf)

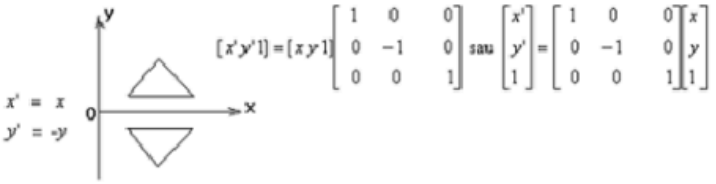
$$[x'\ y'\ 1] = [x\ y\ 1]\underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -x_f & -y_f & 1 \end{bmatrix}}_{T(-xf,\ -yf)}\underbrace{\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{S(0,\ 0,\ sx,\ sy)}\underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x_f & y_f & 1 \end{bmatrix}}_{T(xf,\ yf)}$$

Matricea transf compuse : M = t(-xf, -yf) * S(0,0,sx,sy) * T(xf, yf)
Aplicarea transf compuse : [x' y' 1] = [x y 1] * M

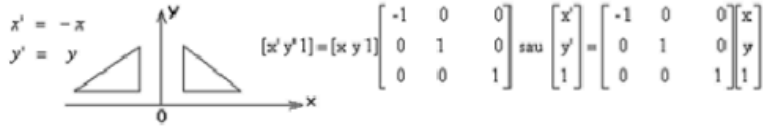
- **OGLINDIREA**

Oglindirea

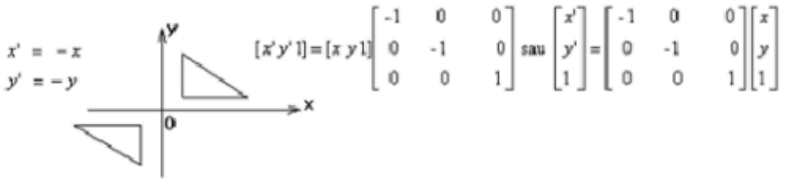
Față de axa OX



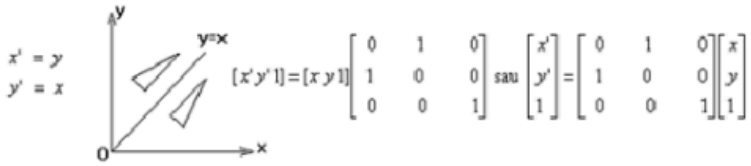
Față de axa OY



Față de origine



Față de dreapta x=y

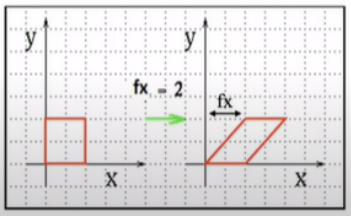


- **OGLINDIREA FATA DE O DREAPTA OARECARE**
  - transformare compusa din :
    - 1 translatie ai. dreapta trece prin origine
    - 2 rotatie fata de origine ai dreapta sa se suprapuna peste OX sau OY
    - 3 oglindire fata de axa principala peste care a fost suprapusa
    - 4 rotatie inversa fata de cea de la pas2
    - 5 translatie inversa fata de cea de la pas1
    - -> **M = T * R * R^(-1) * T^ (-1) folosind vectori linie**
    - **(pasii pentru a construi matricea M se citesc de la dreapta la stanga - 5 4 3 2 1 * [x y 1]**

- **FORFECAREA**
  - definita prin 2 nr reale
    - **Fx , Fy factori de forfecare**
  - transformare compusa din:
    - translatie prin care pct (xf, yf) ajunge in origine
    - forfecare fata de origine
    - translatia inversa

Forfecarea fata de ox

- f_x factor de forfecare de-a lungul axei ox

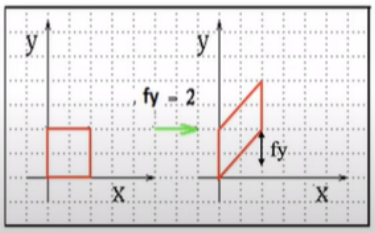


$$x' = x + y * fx$$
$$y' = y$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & f_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Forfecarea fata de axa oy

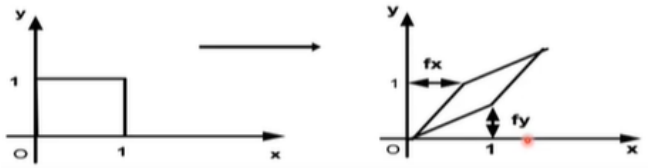
- f_y factor de forfecare de-a lungul axei oy



$$x' = x$$
$$y' = y + x * fy$$

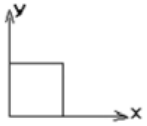
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ f_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Forfecarea fata de ambele axe

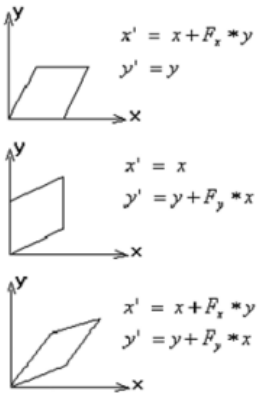


$$x' = x + y * fx$$
$$y' = y + x * fy$$

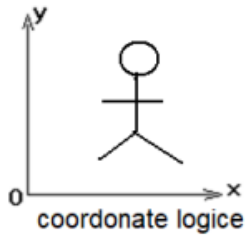
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & f_x & 0 \\ f_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



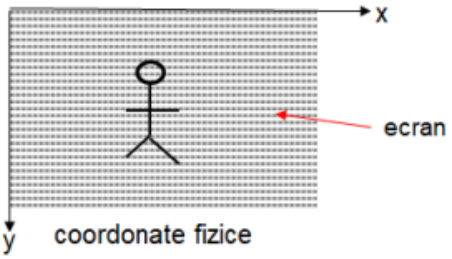
Forfecarea fata de origine



CURS 3
COORDONATE LOGICE SI COORDONATE FIZICE



(metri, milimetri, viteza, timp, etc.)
Sistemul de coordonate utilizator

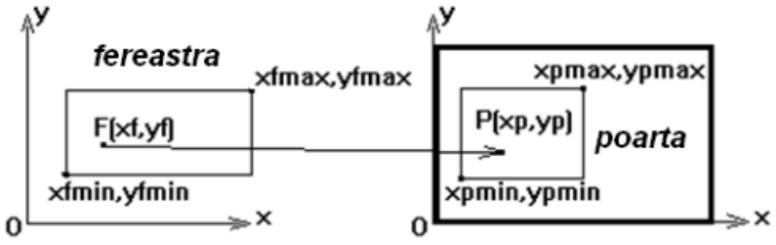


(pixeli, milimetri, s.a.)
Sistemul de coordonate dispozitiv

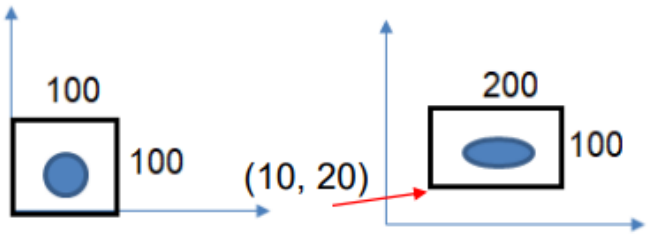
- ! coordonatele logice trebuie transformate in coordonate fizice
- este neccesara o transformare generala prin care un desen definit in coordonate logice sa poata fi afisat intr-o zona oarecare a ecranului

TRANSFORMAREA FEREASTRA-POARTA

- **fereastra =** zona din spatiul coord logice al carei continut se va afisa
- **poarta =** zona din spatiul coord fizice in care se va afisa continutul ferestrei
- transformarea este definita folosind 2 dreptunghiuri, def in cele 2 sisteme de coordonate : fereastra de vizualizare si poarta de afisare
- F : un punct din fereastra
- P : punctul in care se transforma F prin transformarea fereastra-poarta
- trebuie sa se asigure ca **pozitia relativa a lui P in poarta de afisare este aceeasi cu pozitia relativa a lui F in fereastra**
- **sx, sy - factori de scalare ai transformarii**
  - sx = (xp_max - xp_min) / (xf_max - xf_min)
  - sy = (yp_max - yp_min) / (yf_max - yf_min)
- **tx, ty - factori de translatie, depind de pozitiile celor 2 ferestre fata de originea sistemului de coordonate in care sunt definite**
  - tx = xp_min - sx * xf_min
  - ty = yp_min - sy * yf_min
- $$xp = xf * sx + tx$$
  $$yp = yf * sy + ty$$



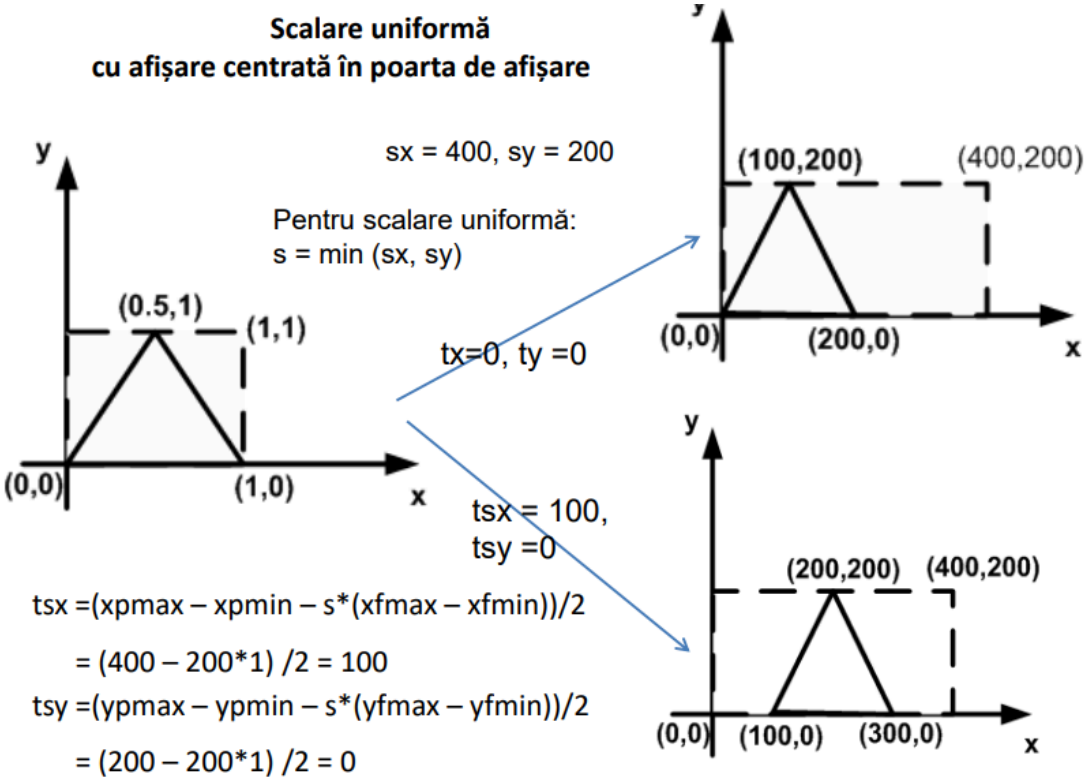
- —> factorii de scalare repr raportul dintre diferentele dimensiunilor maxime/minime ale dreptunghiurilor
- —> coordonatele punctului P se calculeaza in functie de cele ale punctului F pe care il translatam, factorul de scalare si translatia



xf_max = 100	xf_min = 0	yf_max = 100	yf_min = 0
xp_max = 220	xp_min = 10	yp_max = 110	yp_min = 20

sy = yp_max - yp_min / yf_max - yf_min = 120 - 20 / 100 - 0 = 100/100 = 1
sx = xp_max - xp_min / xf_max - xf_min = 210 - 10 / 100 - 0 = 200/100 = 2
tx = xp_min - sx * xf_min = 10 - 2*0 = 10
ty = yp_min - sy * yf_min = 20 - 2*0 = 20

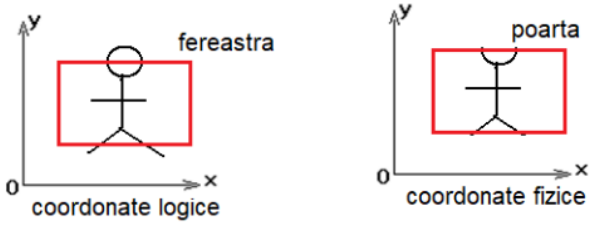
- **SCALARE UNIFORMA CU AFISARE CENTRATA IN POARTA DE AFISARE**
  - pentru scalare uniforma, **s = min (sx, sy)**
  - **tsx = (xp_max - xp_min - s * (xf_max - xf_min)) / 2**
  - **tsy = (yp_max - yp_min - s*(yf_max - yf_min)) / 2**



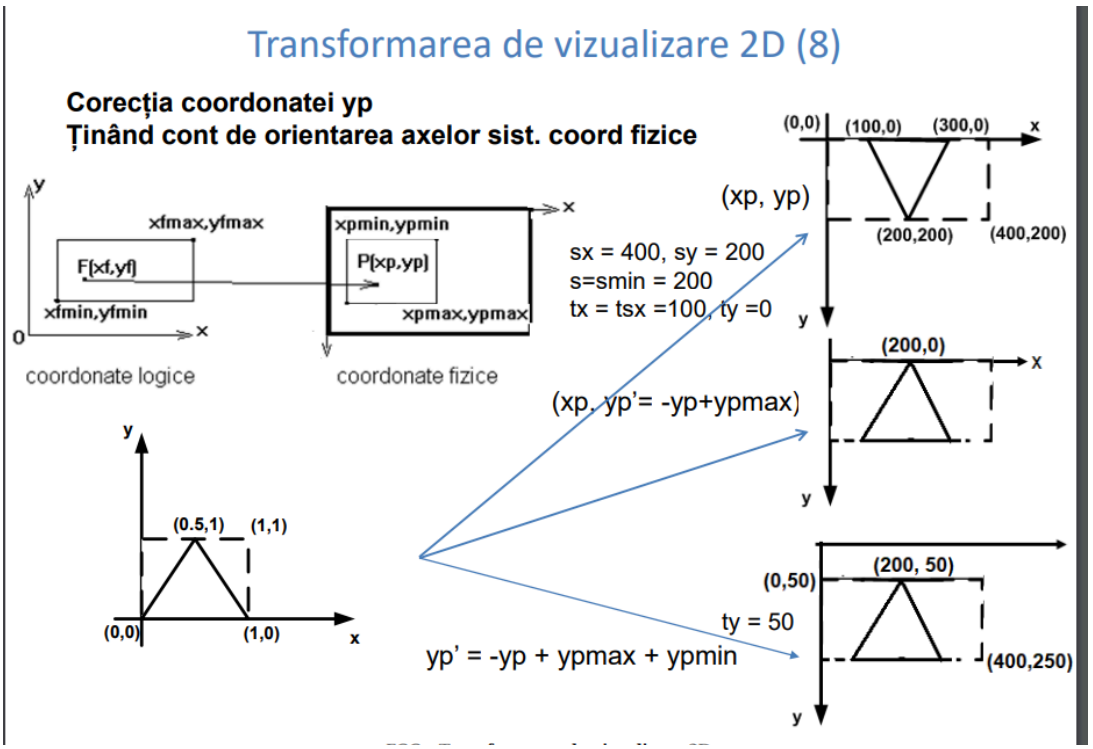
tsx =(xpmax – xpmin – s*(xfmax – xfmin))/2
= (400 – 200*1) /2 = 100
tsy =(ypmax – ypmin – s*(yfmax – yfmin))/2
= (200 – 200*1) /2 = 0

- uniforma = se calculeaza factorul de translatie raportat la dimensiunile dreptunghiurilor, nu raportat la originea planurilor din care fac parte
- **translatia totala : tx + tsx ty + tsy**

- **EFECTELE TRANSFORMARII**
  - **marire/micsorare** in functie de dimensiunile ferestrei si ale portii
  - **deformare** - cand fereastra si poarta nu sunt dreptunghiuri asemenea
  - **pentru scalare uniforma s = min(sx, sy)**
  - **afisare centrata in poarta** - translatie suplimentara pe axa ox sau pe axa oy
    - **tsx = (xp_max - xp_min - s * (xf_max - xf_min)) / 2**
    - **tsy = (yp_max - yp_min - s * (yf_max - yf_min)) / 2**
    - **translatia totala : tx + tsx ty + tsy**
  - **decuparea** primitivelor aflate in afara ferestrei vizuale



- **CORECTIA COORDONATEI YP TINAND CONT DE ORIENTAREA AXELOR SIST COORD FIZICE**

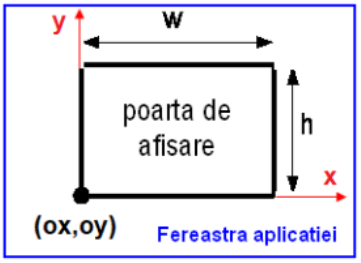


  - deoarece axa OY in coordonate fizice este opusa ca sens celei in coordonate logice coordonata y a punctului translatat va fi cu semn opus, coord x va ramane la fel
  - xp = xf * sx + tx
  - **yp = – (yf * sy + ty) + yp_min + yp_max = -yf * sy + yp_min + yp_max -ty**

$$\begin{bmatrix} xp \\ yp \\ 1 \end{bmatrix} = M * \begin{bmatrix} xf \\ yf \\ 1 \end{bmatrix}$$

$$M = \begin{bmatrix} sx & 0 & tx \\ 0 & -sy & yp\min + yp\max - ty \\ 0 & 0 & 1 \end{bmatrix}$$

- **IMPLEMENTAREA TRANSFORMARII DE VIZUALIZARE IN OPENGL**
  - orientarea axelor viewportului este aceeasi cu a sistemului coordonatelor logice
  - nu este necesara corectia coordonatei y



$$M = \begin{bmatrix} sx & 0 & tx \\ 0 & sy & ty \\ 0 & 0 & 1 \end{bmatrix}$$

CURS 4
TRANSFORMARI GEOMETRICE 3D

- **SISTEM DE COORDONATE CARTEZIENE 3D**
  - **OXYZ**
- **TRANSFORMARILE GEOMETRICE 3D ELEMENTARE**
  - **translatia**
  - **scalarea fata de origine**
  - **rotatiile in jurul axelor principale ale sistemului de coordonate**
  - **oglindirea fata de un plan al sist de coord**
  - **forfecarea fata de originea sist de coord**
- **TRANSLATIA**
  - definita printr-un vector T[tx,ty,tx]
  - x' = x + tx
  - y' = y + ty
  - z' = z + tz
  - matricea $\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$:

$T(t_x, t_y, t_z)$

- **SCALAREA FATA DE ORIGINE**
  - sx = sy = sz -> scalare uniforma
  - orice alta relatie intre factorii de scalare -> scalare neuniforma

$$x' = x \cdot s_x$$
$$y' = y \cdot s_y$$
$$z' = z \cdot s_z$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$S(s_x, s_y, s_z)$

- **ROTATIA POZITIVA (TRIGONOMETRICA) IN JURUL AXEI OZ**
  - rotatie intr-un plan z constant -> nu se modifica coord z
  - **x' = x*cos(u) - y*sin(u)**
  - **y' = x*sin(u) + y*cos(u)**
  - **z' = z**

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$R_z(\theta)$

- **ROTATIA POZITIVA IN JURUL AXEI OX**
  - **y' = y*cos(u) - z*sin(u)**
  - **z' = z*cos(u) + y*sin(u)**
  - **x' = x**

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$R_x(\theta)$

(y se calculeaza ca x in xoy ; z ca y) (plan YOZ)

- **ROTATIA POZITIVA IN JURUL AXEI OY**
  - **z' = z*cos(u) - x*sin(u)**
  - **x' = x*cos(u) + z*sin(u)**
  - **y' = y**

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$R_y(\theta)$

(z ca x in xoy si x ca y) (plan ZOX)

- **OGLINDIREA FATA DE UN PLAN AL SISTEMULUI DE COORDONATE**

$$[O_z] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **FORFECAREA FATA DE ORIGINE**

  De-a lungul axei OZ: modifică x și y proporțional cu z

  x' = x + Fx*z
  y' = y + Fy*z
  z' = z



  Analog pentru forfecarea de-a lungul axei OX si a axei OY

  Cazul general (forfecarea pe toate cele 3 axe):

  x' = x + y*d + z*g
  y' = x*b + y + z*i
  z' = x*c + y*f + z

$$[x'\ y'\ z'\ 1] = [x\ y\ z\ 1] * \begin{vmatrix} 1 & b & c & 0 \\ d & 1 & f & 0 \\ g & i & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

  - **TRANSFORMARI GEOMETRICE 3D COMPUSE**
    - **scalarea fata de un punct oarecare F(xf,yf,zf)**
      - T(xf,yf,zf) * S(sx,sy,sz) * T(-xf, -yf, -zf)
    - **rotatia in jurul unei axe paralele cu o axa a sistemului de coordonate:**

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = M * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

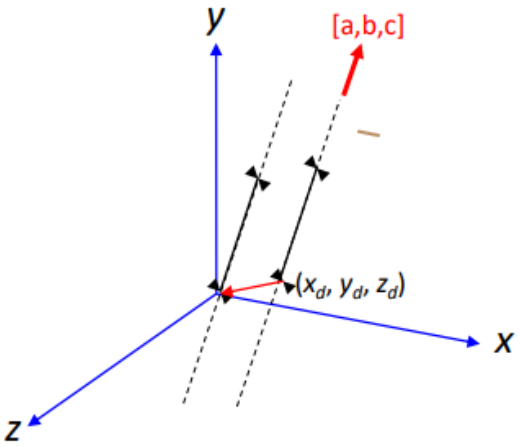
      1. translatie ai axa de rotatie se suprapune cu una din axele sist de coord
      2. rotatia in jurul axei sist de coord peste care s a suprapus
      3. translatia inversa
         -> M = T(xd, yd, zd) * R (u) * T(-xd, -yd,-zd)
         D(xd,yd,zd) un punct de pe axa de rotatie

      - **daca era in jurul unei axe paralele cu OX** -> foloseam matricea de rotatie pozitiva in jurul axei OX

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = T(xc, yc, zc)\, R_x(u)\, T(-xc, -yc, -zc) \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

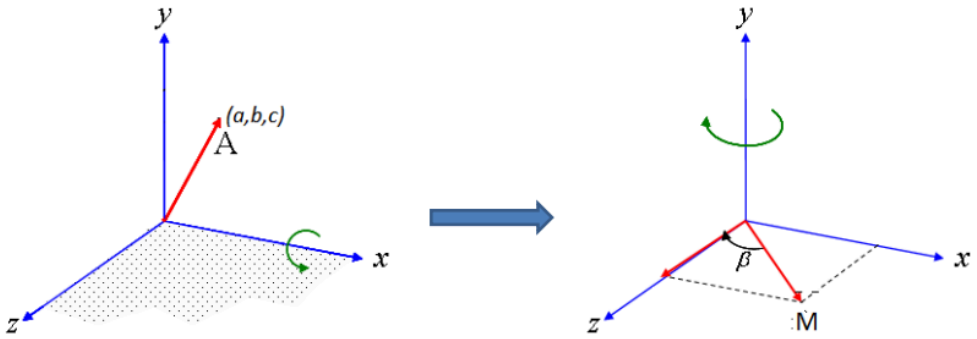
- **rotatia in jurul unei drepte oarecare (xd,yd,zd) si directia sa D[a,b,c]**
  1. translatie prin care dreapta va trece prin origine
  2. aliniere dreapta cu una din axe
  3. rotatia propriu zisa
  4. transformarea inversa de la pas 2
  5. transformarea inversa de la pas 1

1. Translație



$$T = \begin{bmatrix} 1 & 0 & 0 & -xd \\ 0 & 1 & 0 & -yd \\ 0 & 0 & 1 & -zd \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

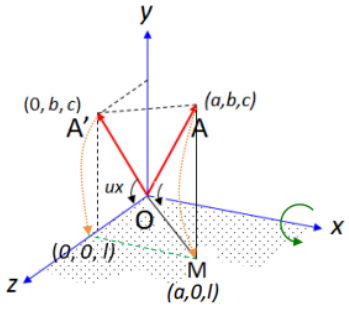
2. Aliniere *D* cu axa OZ



Rotație în jurul axei OX cu un unghi ux, prin care dreapta ajunge în planul XOZ: Rox(ux)

Rotație în jurul axei OY, cu un unghi uy, prin care dreapta se suprapune pe axa OZ: Roy(uy)

2.1. Rotația în jurul axei OX



$R_x(ux)$

A' – proiecția lui A în planul YOZ

Lungime (OA) = L; lungime (OA') = l;

$$L = \sqrt{a^2 + b^2 + c^2}$$

$$l = \sqrt{b^2 + c^2}$$

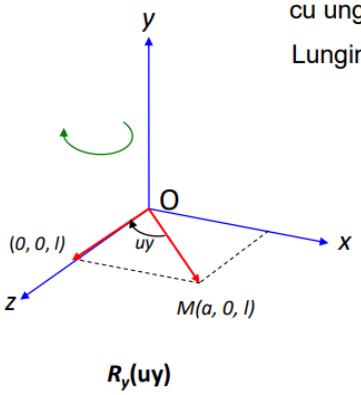
b = l*sin(ux)
c = l*cos(ux)

$$\sin(ux) = \frac{b}{l}$$

$$\cos(ux) = \frac{c}{l}$$

Prin rotația lui OA în jurul axei Ox cu unghiul ux, acesta va ajunge în planul XOZ

2.2. Rotația în jurul axei OY



$R_y(uy)$

M este punctul obținut prin rotația lui A în jurul axei Ox, cu unghiul ux.

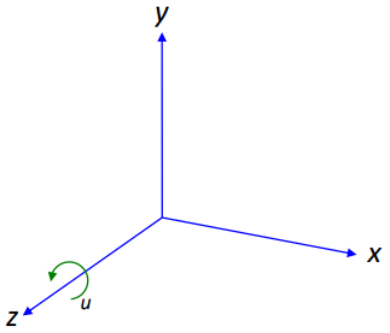
Lungime (OM) = Lungime (OA) = L

a = L*sin(-uy)

l = L*cos(-uy)

$$\sin(-uy) = a/L -> \sin(uy) = -a/L$$
$$\cos(-uy) = \cos(uy) = l/L$$

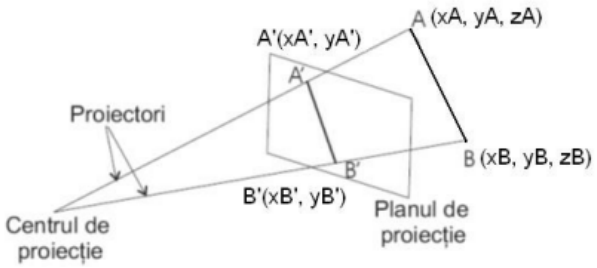
3. Rotația în jurul axei OZ cu unghiul dat



$$R_z(u) = \begin{bmatrix} \cos(u) & -\sin(u) & 0 & 0 \\ \sin(u) & \cos(u) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

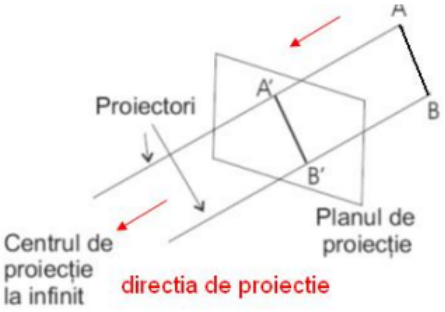
CURS 5
PROIECTII

- **PROIECTII DINTR-UN SPATIU N-DIMENSIONAL IN K-DIMENSIONAL, N>K**
- **PROIECTII R3 -> R2**
- se aplica varfurilor obiectelor
- nu modifica legaturile dintre varfuri
- proiectia unui varf 3d intr-un plan 2d este punctul de intersectie dintre plan si proiectorul care pleaca din **centrul de proiectie** si trece prin varf



Segmentul A'-B' este proiecția segmentului A-B în planul de proiecție XOY (z=0).

- 2 CLASE DE PROIECTII
  - **proiectia perspectiva :** centrul de proiectie e situat la distanta finita fata de planul de proiectie; proiectorii sunt drepte convergente in CP
    (vezi desenul de mai sus)
  - **proiectii paralele :** centrul de proiectie este la infinit; proiectorii sunt linii paralele care trec prin varfurile obiectului proiectat si au directia specificata (vectorul **directie de proiectie)**



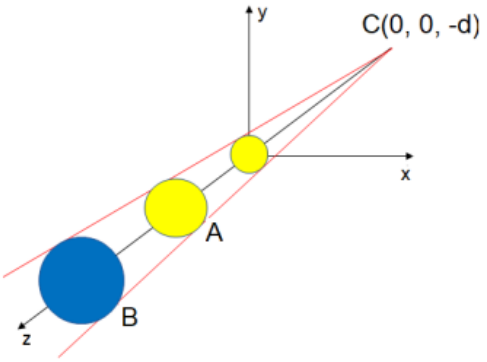
- **PROIECTII PERSPECTIVA**
  - produc imagini asemanatoare cu cele obtinute cu aparatele de fotografiat
  - CARACTERISTICI:
    - **efectul de micsorare**
      - dimensiunea obiectului in proiectia 2d este invers proportionala cu distanta de la centrul de proiectie la obiect

Exemplu:

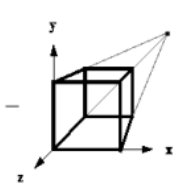
A și B sunt 2 sfere, cu centrele pe axa OZ, care se proiectează în același disc în planul XOY.

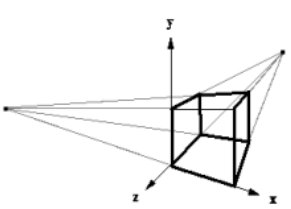
raza sferei B = 2* raza sferei A

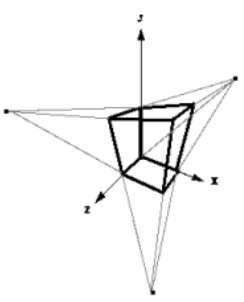
Dist(centru_B-C) = 2* Dist(centru_A-C)



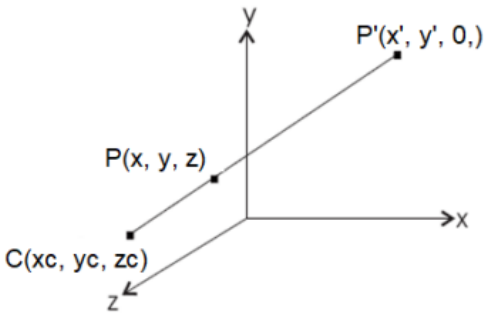
    - **modifica unghiurile dintre dreptele care nu sunt paralele cu planul de proiectie**
      - proiectiile liniilor paralele care nu sunt paralele cu planul de proiectie converg catre un punct din plan**, numit punct de convergenta (**ex : pct C din desenul de mai sus)
  - pot fi efectuate proiectii cu 1, 2 sau 3 puncte de convergenta principale
  - nr de puncte de convergenta principale = nr de axe principale intersectate de planul de proiectie







- **PROIECTIA PERSPECTIVA IN PLANUL XOY**



C – centrul de proiecție
P – punctul proiectat
P' – proiecția lui P în XOY

ecuatiile parametrice ale proiectorului care trece prin P:

x = xc + (x - xc)*t
y = yc + (y - yc)*t 0 <= t <= 1 pt punctele aflate pe segmentul C-P
z = zc + (z - zc)*t

proiectam pe XOY -> z = 0

-> 0 = zc + (z - zc)*t'
-> t' = – zc / (z – zc)
x' = xc + x*t' - xc *t' = xc(1-t')+ x*t' =.....
----------> **x' = (xc * z – zc * x)/(z – zc)**
y' = (yc * z – zc * y)/(z – zc)

! matricea proiectiei depinde de coordonata z a punctului proiectat -> nu poate fi folosita aceeasi matrice pentru toate varfurile aceluiasi obiect
! coord omogene ale punctului obtinut prin proiectie perspectiva au w != 1

Pentru conservarea informației de adâncime (necesară pentru eliminarea din imagine a părților nevizibile ale scenei 3D) se folosește matricea :

Coordonatele omogene ale punctului P'

xw = -zc*x + xc*z → x' = (-zc*x + xc*z)/(z-zc)
yw = -zc*y + yc*z → y' = (-zc*y + yc*z)/(z-zc)
zw = z → z' = z/(z – zc)
w = z – zc

$$P_{perspectiva} = \begin{vmatrix} -zc & 0 & xc & 0 \\ 0 & -zc & yc & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & -zc \end{vmatrix}$$

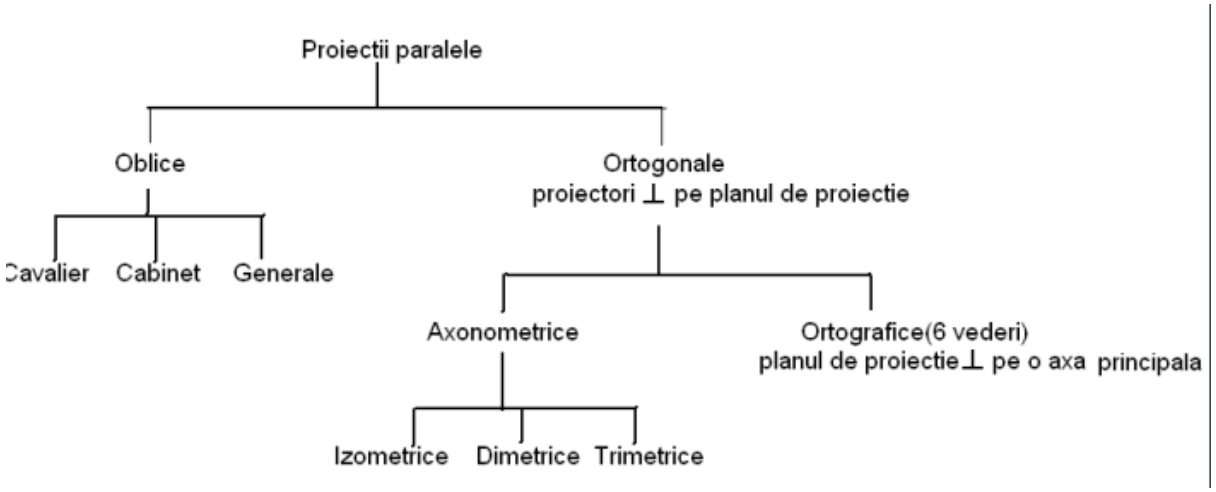
Proiecția perspectivă standard

Xc = 0, Yc = 0, Zc = - d

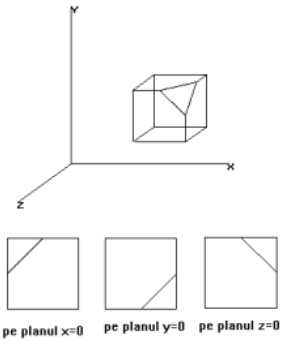
xw = x → x' = x*d/(z+d)
yw = y → y'= y*d/(z+d)
zw = z → z' = z/(z+d)
w = z/d + 1 = (z+d)/d

$$P_{pers_standard} = \begin{vmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \frac{1}{d} & 1 \end{vmatrix}$$

- **PROIECTII PARALELE**
  - CARACTERISTICI
    - **proiectorii sunt drepte paralele de directie data(directia de proiectie)**
    - **sunt transformari afine(conserva paralelismul liniilor**
    - **se conserva unghiurile doar pentru fetele obiectului ce sunt paralele cu planul de pr**
  - proiectii **ortogonale :** proectorii perpendiculari pe plan
    - ortografice - planul de pr || cu un plan principal si perpendic pe o axa principala
    - axonometrice - planul de pr e oarecare
  - proiectii **oblice :** proiectorii nu sunt perpendiculari pe planul de pr
    - cazuri particulare
      - pr Cavalier - unghi 45 grade proiectori-plan
      - pt Cabinet - unghi 63.43 grade



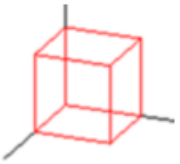
- **PROIECTII ORTOGONALE-ORTOGRAFICE**
  - numite si <u>vederi</u>
  - YOZ - vederea din stanga/dreapta
  - XOZ - vederea de sus/jos
  - XOY - vederea din fata/spate
  - **conserva lungimile laturilor si unghiurile**
  - **utile in desen tehnic**



Proiectii ortografice ale unui cub cu un colt taiat

- **PROIECTII ORTOGONALE-AXIOMETRICE**
  - planul de pr nu e perpendicular pe niciuna din axe
  - **redau mai multe fete ale obiectului proiectat(ca si cele perspectiva)**
  - **alegand planul de pr se poate controla scalarea laturilor**





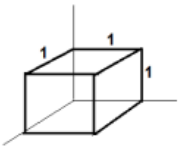
- clasificare in functie de unghiul dintre plan de pr si axele sist de coord:
  - **pr izometrice : unghiuri egale cu toate axele**
    - prin proiectie, laturile sunt scalate cu factori egali
    - **sx = sy = sz**
  - **pr dimetrice : unghiuri egale cu 2 axe**
    - **sx = sy** sau **sx = sz** sau **sy = sz**
  - **pr trimetrice : niciun unghi egal**
    - **sx != sy != sz**

- **PROIECTII OBLICE**
  - planul pr perpendicular pe o axa principala
  - directia de pr nu e perpendiculara pe planul de pr
  - fetele || cu planul de pr se proiecteaza fara sa li se modifice unghiurile/dimensiunile laturilor
  - alegand directia de pr se obtin cazurile particulare cavalier si cabinet

❑ **Proiecții Cavalier**

(unghiul proiectorilor cu planul de proiecție = 45 grade)

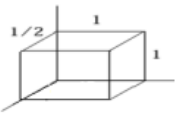
• Lungimea proiecției unei laturi perpendiculare pe plan este egală cu lungimea laturii 3D (conservă lungimea laturilor perpendiculare pe planul de proiecție)



❑ **Proiecții Cabinet**

(unghiul proiectorilor cu planul de proiecție = 63.43 grade)

• Lungimea proiecției unei laturi perpendiculare pe plan este egală cu jumătate din lungimea laturii 3D







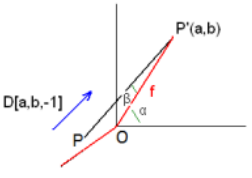
Proiecție Cavalier: f=1, beta=45, alfa=45

Direcția de proiecție: [cos(45),sin(45),-1]

Proiecție Cabinet: f=0.5, beta=63.43, alfa=45

Direcția de proiecție:

[0.5*cos(45),0.5*sin(45),-1]



Reprezentarea matriceală a proiecțiilor paralele:

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Pparalela \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix} \qquad Pparalela = \begin{vmatrix} 1 & 0 & -\frac{a}{c} & 0 \\ 0 & 1 & -\frac{b}{c} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Ca și în cazul proiecțiilor perspectivă, pentru conservarea informației de adâncime se folosește matricea →

$$Pparalela = \begin{vmatrix} 1 & 0 & -\frac{a}{c} & 0 \\ 0 & 1 & -\frac{b}{c} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

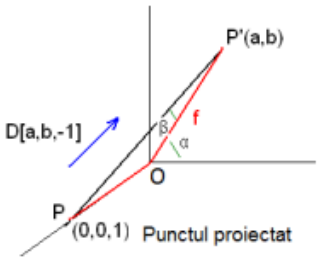
Cazuri particulare

1. Proiecția ortografică în XOY (proiectorii perpendiculari pe XOY):

D[a, b, c]
a= 0, b = 0 →

$$P_{O\text{-}XOY} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2. Proiecții oblice:



c = -1 →

$$P_{OBI} = \begin{bmatrix} 1 & 0 & a & 0 \\ 0 & 1 & b & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$P_{OBI} = \begin{bmatrix} 1 & 0 & f*\cos(\alpha) & 0 \\ 0 & 1 & f*\sin(\alpha) & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

OP de lungime 1 se proiectează în OP' de lungime f: → f este factorul de scalare al laturilor perpendiculare pe XOY (OP este perpendicular pe XOY).

tg(β) = 1/f

f = 0 → β=90^0 : proiectie ortografica

f = 1 → β=45^0 : proiectie Cavalier

f = 0.5 → β=63.43^0 : proiectie Cabinet

α este un parametru liber

in mod uzual, α= 30^0 sau 45^0

- **PROIECTII AXONOMETRICE**
  - 2 POSIBILITATI DE EXPRIMARE MATEMATICA:
  - obiect fix in spațiu, alegere plan astfel încât să obținem cele 3 cazuri (izometrice, dimetrice, generale):

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Pax(N,R) \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

  - plan de proiecție fix (XOY), poziționare obiect in spațiu:
  - TA = rotatii in jurul axelor OX,OY

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Po\text{-}xoy * TA \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

Proiecții axonometrice (2)

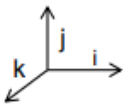
Exemplu:

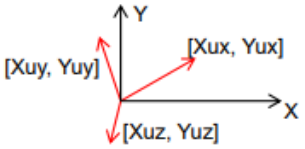
TA = Rx(ux) * Ry(uy)

PA = Po-xoy* TA = $\begin{bmatrix} \cos(uy) & 0 & \sin(uy) & 0 \\ \sin(ux)^*\sin(uy) & \cos(ux) & -\cos(uy)^*\sin(ux) & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

Se aplică transformarea PA versorilor axelor principale:

$$\begin{bmatrix} Xux & Xuy & Xuz \\ Yux & Yuy & Yuz \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} = PA \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$





Factorii de scalare a laturilor pe cele 3 axe sunt lungimile proiecțiilor celor trei vectori unitate:

$sx = (Xux^2 + Yux^2)^{0.5}$

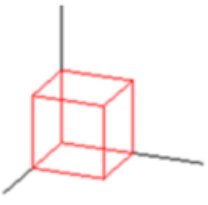
$sy = (Xuy^2 + Yuy^2)^{0.5}$

$sz = (Xuz^2 + Yuz^2)^{0.5}$

Se deduc unghiurile de rotație în jurul axelor pentru obținerea celor 2 cazuri particulare de proiecții axonometrice: izometrice și dimetrice

- **proiectii izometrice :**
  - **sx = sy = sz**
  - **ux = +/- 35.26 grade**
  - **uy = +/- 45 grade**
- **proiectii dimetrice**
  - **<u>in cazul sx = sy</u>**
  - **ux = arcsin(+- sz / v2)**
  - **uy = arcsin(+/- sz/ v (2 - sz^2))**
  - sz poate fi ales intre 0 si 1
  - pentru fiecare sz exista 4 proiectii dimetrice, de exemplu
  - pentru sz = 0.5 : ux = +/- 20.705 grade, uy = +/- 22.208 grade





Proiecție izometrică:

P_ort_XOY * R_Ox(35.26)* R_Oy(45)

sx = sy = sz

Proiecție dimetrică:

P_ort_XOY * R_Ox(20.705)* R_Oy(22.208)

sx = sy # sz

CURS 6
TRANSFORMAREA DE VIZUALIZARE DIN SPATIUL 3D

- scop : obtinerea unei vederi a modelului unei scene 3d pe o suprafata de afisare
- compusa din mai multe transformari, printre care si o proiectie R3->R2
- transf se efectueaza asupra varfurilor obiectelor
- include si operatia de decupare a primitivelor la marginea volumului vizual
- transf de **modelare** (matricea **M**) -> **vizualizare** (matricea **V**) -> **proiectie** (matricea **P**) -> impartirea perspectiva -> transf in poarta de afisare
- **MV = V * M (matricea model-view)**
- **MVP = P * V**

  Transformarea vârfurilor din coordonate obiect în coordonate de decupare (clip):

  $$[xc, yc, zc]^T = MVP \cdot [xo, yo, zo]^T$$

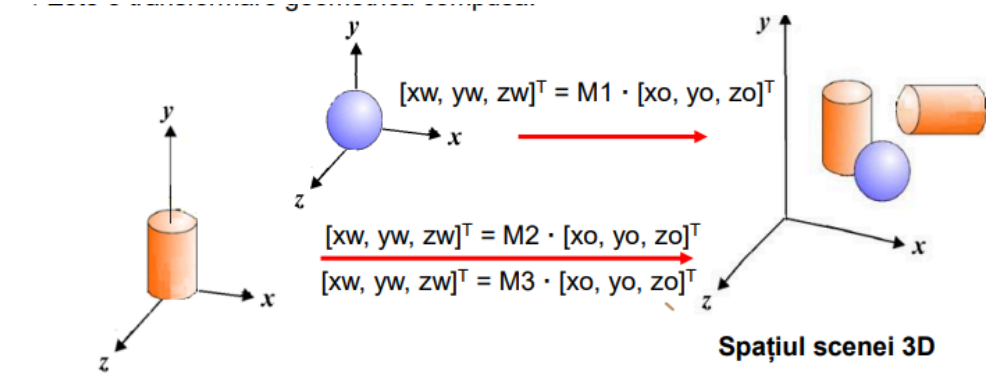
  Matricile sunt calculate în programul OpenGL folosind functii ale bibliotecii **glm** şi transferate la GPU

  Vârfurile sunt transformate în programul ***Vertex Shader executat pe GPU,***

  folosindu-se matricea MVP

  Dupa asamblarea primitivelor, folosindu-se vârfurile transformate şi topologia transmisă la GPU de programul OpenGL, primitivele sunt decupate la frontiera volumului vizual.

- **TRANSFORMAREA DE MODELARE**
  - **spatiul obiect (sist coord obiect)**
    - fiecare obiectb are propriul sist de coord
    - ex. cubul, sfera, schelet personaj ..
    - obiectul e definit de regula centrat in originea sistemului sau de coordonate
  - **spatiul scenei 3D (sist coord globale)**
    - spatiul in care e compusa scena 3d
    - raportat la un sistem de referinta global - sist de coordonate carteziene 3d “dreapta”
    - scena redata intr un cadru imagine e reprez in sistemul coord globale
  - scop : **dimensionarea** si **pozitionarea** obiectelor scenei intr-un cadru imagine
  - este **o transformare geometrica compusa**



Coordonatele obiect ale vârfurilor obiectelor sunt transformate pentru modelarea scenei 3D

Transformarea de modelare

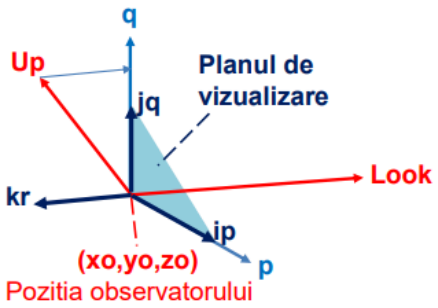
↓

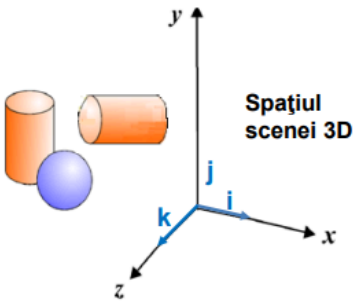
Rotaţie, Scalare, Translaţie

Obiecte definite în sistemul coordonatelor globale

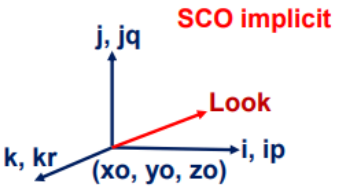
- **TRANSFORMAREA DE VIZUALIZARE**
  - **sist coord globale -> sist coord observator**
  - imagine = vedere dintr-un punct al spatiului 3d
  - depinde de:
    - **pozitia camerei in spatiul lumii reale**
    - **orientarea camerei(**directia in care priveste observatorul si rotatia camerei fata de axa sa)
  - spatiul observator = spatiul 3d vazut din poz obs pe directia in care priveste
  - obiectele ce apartin spatiului obs sunt raportate la un sist de coord cu originea in pozitia obs (camerei virtuale)
  - **sistemul de coord observator e definit prin 3 parametrii:**
    - **pozitia camerei virtuale (**originea x0,y0,z0)
    - **directia in care priveste observatorul -** normala la planul de vizualizare
    - **directia "sus" - directia axei oy a sist = <u>rotatia camerei in jurul axei privirii(look)</u>**

Sistemul de coordonate observator



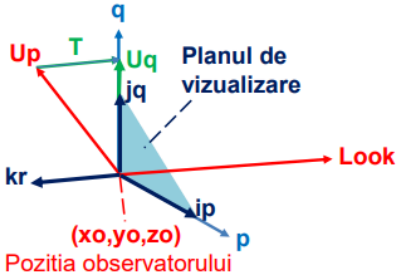


- (xo, yo, zo): originea sistemului de coordonate observator (SCO)
- Look – normala la planul de vizualizare
- axa q: axa verticală a planului de vizualizare – are direcția data de proiecția vectorului UP în planul de vizualizare
- (ip,jq,kr) formează un sistem de coordonate 3D dreapta



ip, jq, kr: versorii axelor sistemului de coordonate observator coincid cu cei ai sist. coordonatelor globale

Determinarea axelor sistemului de coordonate observator



kr = - Look/|Look|

Uq: proiectia ortogonala a vectorului Up in planul de vizualizare

Uq dă direcţia axei verticale a planului de vizualizare

Uq = Up + T

T || (-kr) → T = s*(-kr) = - s*kr (s un scalar oarecare) → Uq = Up - s*kr

Uq perpendicular pe kr → Uq·kr = 0 → (Up - s*kr)·kr =0 → Up·kr - s*kr·kr = 0 → s = Up·kr

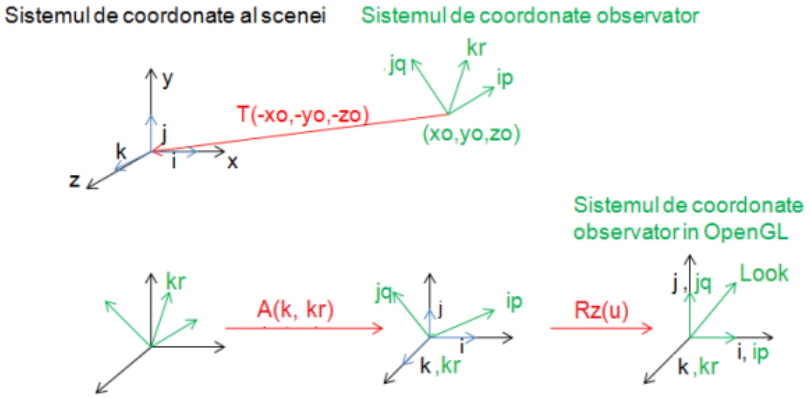
Uq = Up – (Up·kr)·kr

jq = Uq/|Uq| este versorul axei verticale a planului de vizualizare

ip = (jq x kr) / |(jq x kr)| este versorul axei orizontale a planului de vizualizare

(ip, jq, kr) formează un sistem de coordonate carteziene 3D dreapta

- **calculul matricei transformarii de vizualizare :**
  - sist de coord observator implicit in OpenGl considera:
    - **pov : originea sist de coord globale**
    - **directia in care priveste : dir negativa a axei OZ**
    - **directia sus a planului de viz : dir pozitiva OY**
  - cu aceste valori implicite -> **transformarea de vizualizare e transformarea identica**
  - pentru calculul matricei transf de vizualizare putem folosi functia **lookAt(poz obs, pct spre care priveste obs, vectorul look, vectorul up)**

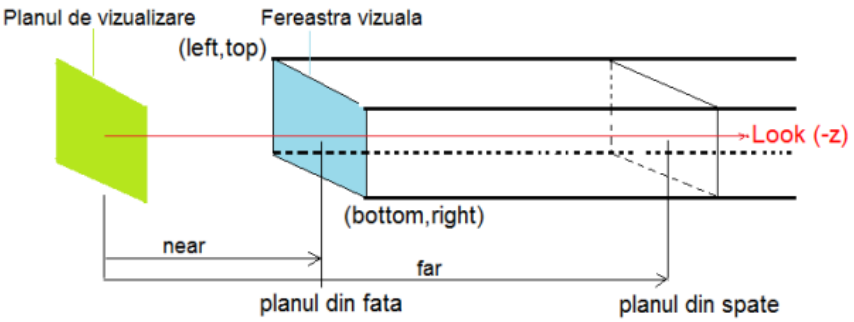


- A(k,kr) este o transformare prin care se aliniază axele z ale celor 2 sisteme de coordonate: constă din 2 rotații – o rotație în jurul axei X și una în jurul axei Y.
- Prin rotația în jurul axei z (Rz) se aliniază celelalte 2 axe.

Matricea transformării de vizualizare: V = Rz(u) * A(k,kr) * T(-xo,-yo,-zo)

- **se aplica transpunere catre originea sist global, 2 rotatii(in jurul axei OX si OY) ca sa suprapunem axele OZ si inca o rotatie in jurul OZ ca sa suprapunem ox si oy de la cele doua sist**

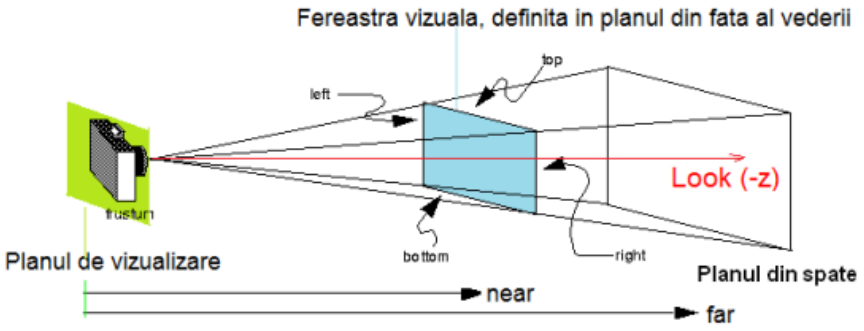
- **TRANSFORMAREA DE MODELARE SI VIZUALIZARE**
  - **sist coord obiect -> sist coord observator**
  - TRANSFORMARE COMPUSA MODEL-VIEW (MV = V * M)
  - Transformarea vârfurilor din coordonate obiect în coordonate observator **(eye): [xe ye ze we]T = MV • [xo yo zo wo]T**
  - pentru calculele de iluminare a obiectelor se folosesc vectori normali atasati varfurilor obiectelor
- **TRANSFORMAREA DE PROIECTIE**
  - determinata de **volumul vizual** definit de programator
  - **Volumul vizual al proiectiei ortografice :**



- **coord colturilor ferestrei vizuale sunt raportate la sist coord planului din fata, care are axele paralele cu cele ale planului de vizualizare**

  - Se efectuează proiecția ortografică în fereastra vizuală a obiectelor cuprinse în volumul vizual. Primitivele grafice situate complet in afara volumului vizual sunt eliminate iar cele care intersecteaza volumul sunt decupate la frontiera volumului creându-se noi primitive.

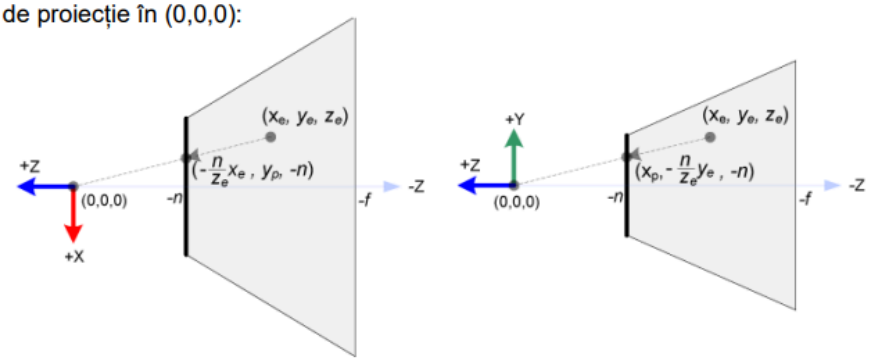
- **Volumul vizual al proiectiei perspectiva (View frustum)**
  - = trunchi de piramida delimitat in adancime de planul din fata(near) si planul din spate(far) al vederii, cu varful in pozitia observatorului



  - Se efectuează proiecția perspectivă în planul din faţă (în fereastra vizuală), cu centrul de proiecţie în poziţia observatorului, a obiectelor cuprinse în volumul vizual. Obiectele situate în afara volumului vizual sunt eliminate sau decupate la frontiera volumului vizual.

❖ **Proiecția se efectuează în fereastra vizuală din planul din** faţă: planul z =-near (in SCO).

Proiecția perspectivă a unui punct din SCO, (xe,ye,ze), pe planul din față al vederii, cu centrul de proiecție în (0,0,0):



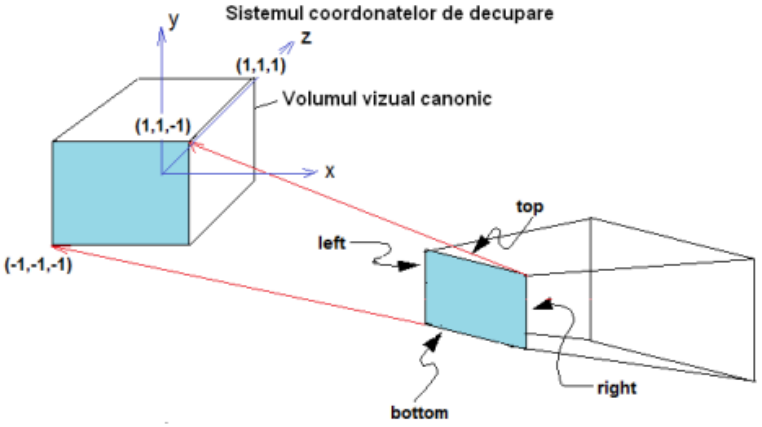
xp = (-near/ze)*xe, yp =(-near/ze)*ye – se pot obține aplicând formulele proiecției perspectivă cu centrul de proiecție în (0,0,0) și planul de proiectie este z=-near.

Proiectia ortografică a unui punct din sist. coordonatelor observator, (xe,ye,ze), în fereastra vizuală:

xp = xe, yp = ye, zp = -near.

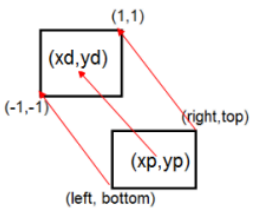
- **transformarea de proiectie este insotita de operatia de decupare a primitivelor grafice la marginea volumului vizual**
- **volumul vizual definit de programator este transformat in volumul vizual canonic prin transformarea de normalizare(**cub cu latura de 2 unitati, centrat in originea sist coordonator de decupare, cu laturile paralele cu axele)
- În OpenGL transformarea de proiecție include: proiecția definită prin volumul de vizualizare compusă cu transformarea de normalizare. = transformarea coordonatelor observator în coordonate de decupare.
- După transformarea varfurilor prin transformarea de proiectie, vârfurile sunt unite pe baza topologiei definite de programator, formând primitive grafice 3D. Acestea sunt decupate la marginile volumului vizual canonic.

- **Sistemul coordonatelor de decupare**: sistem de coordonate carteziene 3D stânga.
- Prin transformarea de normalizare:
  - colţurile ferestrei 2D din planul de proiecţie, (left, bottom,-near) şi (right, top,-near) sunt mapate pe colţurile stânga-jos şi dreapta-sus ale ferestrei 2D din sistemul coordonatelor de decupare, adică (-1,-1,-1) şi (1,1,-1);
  - planul z= -near se mapează pe z= -1, iar z= -far se mapează pe z= 1



Transformarea de proiecţie (7) calculul coordonatelor de decupare

- Impunând condiția (left,bottom) → (-1,-1) și (right,top) → (1,1), rezultă transformarea punctelor din planul din față, (xp,yp), în sistemul coordonatelor de decupare – este o transformare freastră-poartă:



xd = 2*xp/(right – left) – (right + left)/(right – left)

yd = 2*yp/(top – bottom) – (top + bottom)/(top-bottom)

Înlocuind xp și yp rezultate din proiecția perspectivă în planul -near:

xd = (xe*2*near/(right – left) + ze*(right + left)/(right – left))/(-ze)

yd = (ye*2*near/(top – bottom) + ze*(top + bottom)/(top-bottom))/(-ze)

Pentru transformarea coordonatelor z se impun condițiile: -near → -1, -far → 1; rezultă:

zd = (-ze*(far+near)/(far-near) – 2*far*near/(far-near)) / (-ze)

(xd,yd,zd) sunt coordonate carteziene în sistemul coordonatelor de decupare (clip coordinates); folosind matricea transformării de proiecție rezultă coordonate omogene:

[xc, yc, zc, w] → xd = xc/w; yd = yc/w; zd = zc/w ; w = - ze

Obs: w trebuie sa fie ≠ 0 → punctul proiectat nu poate fi în planul camerei (centrul de proi.)

Matricea transformării de proiecție pentru proiecția perspectivă:

$$P = \begin{bmatrix} \frac{2*near}{right-left} & 0 & A & 0 \\ 0 & \frac{2*near}{top-bottom} & B & 0 \\ 0 & 0 & C & D \\ 0 & 0 & -1 & 0 \end{bmatrix} \quad A = \frac{right+left}{right-left} \quad B = \frac{top+bottom}{top-bottom}$$

$$C = -\frac{far+near}{far-near} \quad D = -\frac{2*far*near}{far-near}$$

Matricea transformării de proiecție pentru proiecția ortografică:

xp=xe si yp=ye → xc = 2*xe/(right – left) – (right + left)/(right – left)

yc = 2*ye/(top – bottom) – (top + bottom)/(top-bottom)

$$P = \begin{bmatrix} \frac{2}{right-left} & 0 & 0 & -A \\ 0 & \frac{2}{top-bottom} & 0 & -B \\ 0 & 0 & \frac{-2}{far-near} & C \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[xc\ yc\ zc\ w]^T = P \bullet [xe\ ye\ ze\ we]^T$$

21

Matricile de proiecție pot fi obținute apelând următoarele funcții din biblioteca glm:

glm::ortho(float left, float right, float bottom, float top, float zNear, float zFar);

glm::frustum (float left, float right, float bottom, float top, float zNear, float zFar);

glm::perspective (float fov, float aspect, float znear, float zfar);

Crează o matrice de proiecție pentru un volum perspectivă simetric:

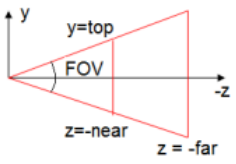
- fov: Field of View

  tan(FOV/2) = top/near

- aspect: aspect ratio

  aspect ratio = (right-left)/(top-bottom)

  = raportul dintre lățimea și înălțimea ferestrei vizuale



(xo, yo, zo): **poziția observatorului**

(Px,Py,Pz): punctul din scena 3D către care privește observatorul;

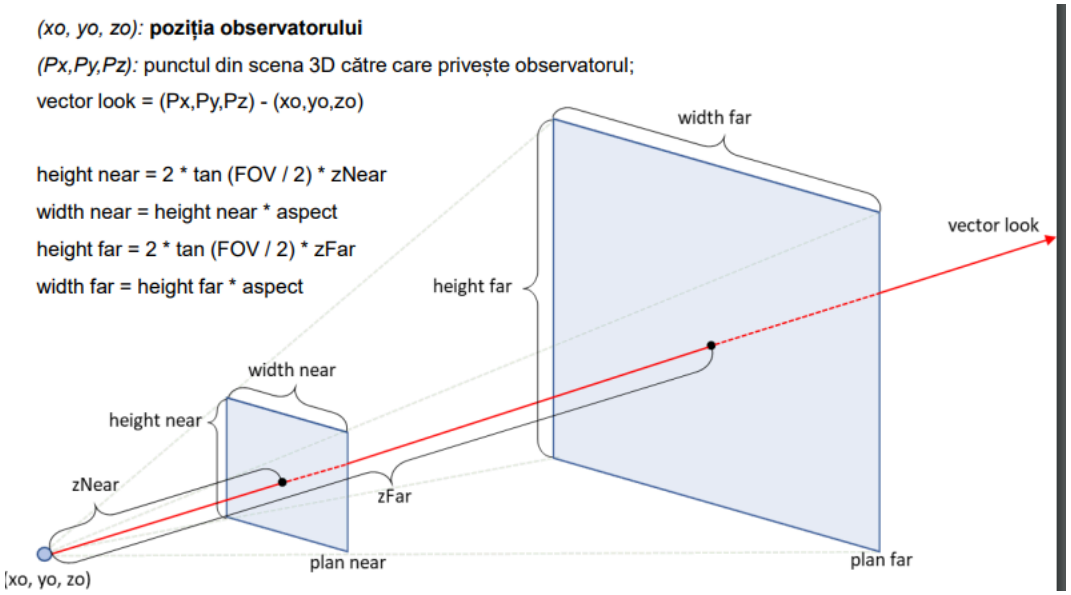
vector look = (Px,Py,Pz) - (xo,yo,zo)

height near = 2 * tan (FOV / 2) * zNear

width near = height near * aspect

height far = 2 * tan (FOV / 2) * zFar

width far = height far * aspect



din coordonate observator → în coordonate de decupare

$[xc\ yc\ zc\ w]^T = P \cdot [xe\ ye\ ze\ we]^T$ (clip coordinates) ← eye coordinates)

Împărţirea perspectivă

Coordonate omogene → coordonate carteziene: xd = xc/w , yd = yc/w , zd= zc/w

(xd, yd, zd): coordonate dispozitiv normalizate

- **Vârfurile rezultate prin împărțirea perspectivă sunt asamblate în primitive grafice care sunt decupate la frontiera volumului vizual canonic.**

Transformarea de modelare-vizualizare-proiecţie

❖ Prin înmulțirea matricilor transformarilor de modelare, vizualizare și proiecție se obține matricea numită "model-view-projection", a.i. asupra vârfurilor definite în sistemul coordonatelor obiect poate fi efectuată o singură transformare:

MVP = P•V•M (matricea ModelViewProjection)

Transformarea in poarta de afişare

Este o transformare fereastră-poartă, în care:

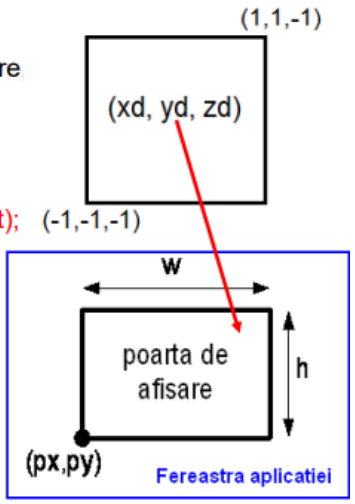
Fereastra este fereastra 2D din sistemul coordonatelor de decupare având colţurile în (-1, -1, -1) - (1, 1, -1).

Poartă este definită de programator folosind funcția:

void **glViewport**(GLint Px, GLint Py, GLsizei width, GLsizei height);

(*Px, Py*) reprezintă coordonate relative la colțul stânga-jos al ferestrei aplicației (în pixeli). Valorile implicite : (0,0).

width, height reprezintă lăţimea / înălţimea porţii de afişare. Valorile implicite sunt: lăţimea şi înălţimea ferestrei aplicației.



Transformarea în poarta de afișare este definită astfel:

xv = (width/2)*xd + ox , ox = px+ width/2

yv = (height/2)*yd + oy, oy = py + height/2

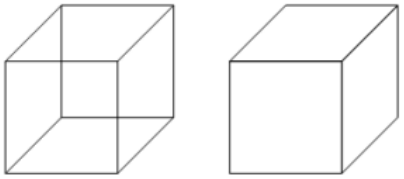
zv = ((f-n)/2)*zd + (n+f)/2 // conservă coordonata z pentru eliminarea părților nevizibile

n (near) şi f (far) au valorile implicite 0.0 respectiv 1.0, pentru care 0<=zv<=1

n și f pot fi modificate la valori cuprinse în intervalul [0,1] folosind funcţia ***DepthRange()***

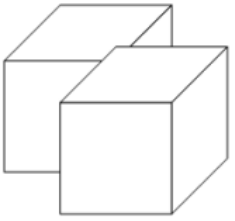
CURS 7
ELIMINAREA PARTILOR NEVIZIBILE ALE SCENELOR 3D DIN IMAGINI

- **vizibilitatea obiectelor dintr-o scena 3d depinde de:**
  - poz observator si directia in care priveste
  - volumul vizual
  - pozitionarea obiectelor din scena unul fata de celalalt

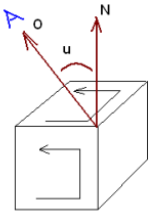


- **partile unei scene 3d nevizibile intr-o imagine sunt**
  - parti aflate total sau partial in afara volumului vizual
  - fete auto-obturate
  - parti obturate de obiecte ale scenei aflate in fata lor in raport cu observatorul

t:

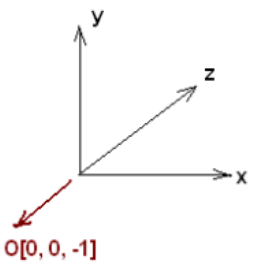


- eliminarea se efectueaza la nivel de:
  - obiecte/ grupuri de obiecte - **object culling**
    - **frustum culling -** eliminarea din banda grafica a obiectelor a obiectelor sau grupurilor de obiecte care sunt complet in afara volumului vizual
    - **occlusion culling -** eliminarea din banda grafica a obiectelor sau grupurilor care sunt complet obturate de alte obiecte
    - nu este efectuata de OpenGL/GPU
    - poate fi efectuata prin algoritmi implementati intr-o biblioteca folosita de aplicatia grafica 3d(motorul grafic)
  - fete ale obiectelor - **back face culling** - eliminarea din banda grafica a fetelor auto-obturate ale obiectelor (este efectuata de GPU)
  - primitive grafice (poligoane, linii)
    - eliminarea primitivelor aflate total sau partial in afara volumului vizual prin operatia de decupare(**clipping**) efectuata ded GPU
    - eliminarea partilor obturate ale primitivelor grafice (**hidden surface/ hidden line removal**)
      - prin algoritmi implementati intr-o biblioteca folosita de aplicatia grafica 3d : **BSP, Pictorului ..**
      - este efectuata de GPU ca operatie raster la nivel de fragment (**algoritmul z-buffer)**
- **Back face culling pentru poliedre complexe:**
  - poate reduce substantial numarul de poligoane rasterizate
  - algoritmul:
    - observatorul si obiectul sunt raportati la acelasi sistem de coordonate
    - conturul fiecarei fete a obiectului este orientat in sens trigonometric atunci cand poliedrul este vazut din exterior
    - observatorul este situat in afara obiectului
    - **fata e vizibila daca 0 <= u <= 90grade sau 0<cos(u)<=1**
    - **conditia de vizibilitate Nu*Ou**



- **Determinarea fetelor auto-obturate poate fi efectuată:**
  - in sist de coordonate globale in care e definit si observatorul-camera virtuala
  - in sist de coordonate observator
  - in sist coord de decupare (dupa aplicarea transformarii de proiectie)

În cazul 3



Sistemul coordonatelor de decupare

Nu • Ou = [nx, ny, nz] • [0, 0, -1] = - nz

Condiția de vizibilitate devine: nz <0

După transformarea de proiecție poziția observatorului este la infinit, pe axa z negativă.

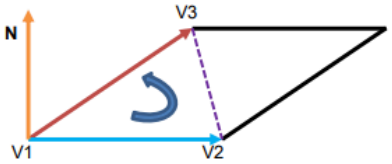
Vectorul orientat către observator este același pentru orice punct al unui obiect

  - **calculul normalei la o fata a poliedrului**

Fie V1(x1, y1, z1), V2(x2, y2, z2), V3(x3, y3, z3), 3 varfuri succesive ale conturului feţei.
Normala la față se poate obține calculând produsul vectorial: (V2-V1)x(V3-V1)

$$\mathbf{N} = (V2-V1)\times(V3-V1) = \begin{vmatrix} i & j & k \\ x2-x1 & y2-y1 & z2-z1 \\ x3-x1 & y3-y1 & z3-z1 \end{vmatrix} = [(y2-y1)*(z3-z1) - (z2-z1)*(y3-y1)]*\mathbf{i} - [(x2-x1)*(z3-z1) - (z2-z1)*(x3-x1)]*\mathbf{j} + [(x2-x1)*(y3-y1) - (y2-y1)*(x3-x1)]*\mathbf{k}$$

unde **i, j, k** sunt versorii direcțiilor axelor sistemului de coordonate carteziene 3D



Normalele:

  - **normalele:**
    - pot fi calculate in sist coordonatelor obiect si atasate in varfuri in OpenGL -> sunt transformate in sist coord de decupare, in vertex shdaer
    - nu sunt atasate varfurilor : sunt calculate de GPU dupa asamblarea primitivelor, folosind varfurile transformate in sist coord de decupare
  - **pentru eliminarea fetelor auto-obturate se apeleaza urmatoarele functii OpenGL**
    - glCullFace(GL_FRONT/GL_BACK/GL_FRONT_AND_BACK)
      - front and back -> nu vor mai fi afisate fete, ci numai alte primitive (linii, puncte)
    - glFrontFace(GL_CCW/GL_CW)
      - specifica orientarea fetelor din fata : counterclockwise/clockwise

➢ Dacă ciclul de vârfuri al fiecărei fețe este orientat trigonometric atunci când obiectul este privit din exteriorul său:

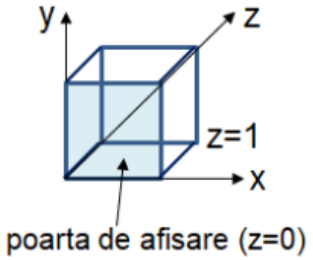
- atunci când observatorul este în spațiul exterior obiectului se va apela glFrontFace(GL_CCW)
- atunci când observatorul este în spațiul interior obiectului se va apela glFrontFace(GL_CW)

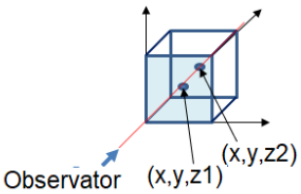
(văzute din interior, fețele obiectului au conturul orientat anti-trigonometric).

- **ALGORITMUL Z-BUFFER**
  - pt eliminarea fragmentelor nevizibile ale primitivelor grafice
  - executat de GPU, alg integrat in procesul de rasterizare a primitivelor

- Observatorul este la infinit, pe axa z negativă

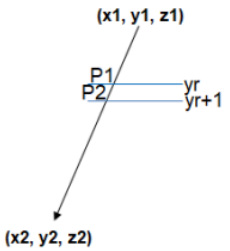
- Asupra primitivelor se efectuează proiecție ortografică în planul XOY (poarta de afișare)





  - daca doua fragmente aflate pe acelasi proiector, rezultate din rasterizarea a doua primitive diferite, au coordonata z diferita:
    - fragmentul care are coord z mai mica va fi afisat in pixelul (x,y)
  - primitivele grafice in care a fost descompusa scena 3d sunt rasterizate in ordinea in care au fost transmise din programul de aplicație (nu sunt sortate)
  - frame buffer - actualizat pe masura rasterizarii primitivelor astfel:
    - **daca fragmentul curent se proiecteaza in pixel(x,y) si z-fragment < z-fragment afisat in pixel(x,y) atunci:**
    - **actualizeaza z-fragment afisat in pixel(x,y), ia valoarea z a fragm curent, actualizeaza culoarea pixelului (x,y) in bufferul imagine, ia culoarea fragmentului curent**
  - **z-buffer =** tablou bidimensional cu nr de elemente egal cu nr de pixeli ai suprafetei de afisare, stocat in memoria GPU
  - PASI:
  - **initializeaza buffer-ul imagine la culoarea de fond**
  - **initializeaza z-buffer la coordonata z maxima (z=1)**
  - pentru fiecare fragm f(x,y,z) rezultat din rasterizarea unei primitive
  - **calculeaza culoarea fram(in fragment shader)**
  - **testul de vizibilitate(adancime) z < z-buffer[y][x] -> actualizeaza culoarea pixelului x,y in bufferul imagine folosind culoarea fragmentului f**

Calculul coordonatei z la rasterizarea vectorilor



x = x1 + t(x2-x1)
y = y1 + t(y2-y1) Ecuațiile parametrice ale vectorului
z = z1 + t(z2-z1)

yr, yr+1: ordonatele a 2 linii raster (ecran) care intersectează vectorul

y(P1) = yr = y1 + t'(y2-y1) → t' = (yr-y1)/(y2-y1)

z(P1) = z1 + (yr-y1)(z2-z1)/(y2-y1) = z1 + (yr-y1)*K,
K este o constantă la rasterizarea vectorului

z(P2) = z1 + (yr+1-y1)*K = z1 + (yr-y1)*K + K

Z(P2) = Z(P1) + K → coordonatele z ale fragmentelor de pe vector se obțin prin calcul incremental.

Calculul coordonatei z la rasterizarea poligoanelor

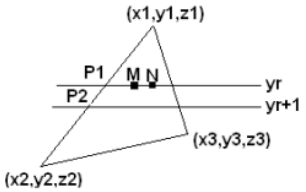
P1(xP1, yr, zP1), P2(xP2, yr+1, zP2)

m- panta laturii (x1,y1,z1) – (x2,y2,z2)

xP2 = xP1 + 1/m (vezi curs - rasterizare)

A*x + B*y + C*z + D = 0 ec. planului poligonului

C*z = -A*x - B*y - D



zP1 = (-A*xP1 - B*yr –D)/C

zP2 = (-A*xP2 - B*(yr +1) – D)/C = (-A(xP1 + 1/m) - B*yr –B – D)/C = zP1 + (-A/m –B) → **zP2 = zP1 + K1**

M(xM, yr, zM), N(xM + 1, yr, zN)

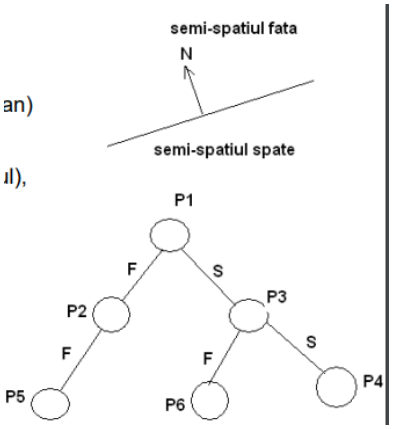
zM = (-A*xM – B*yr – D)/ C

zN = (-A*(xM + 1) – B*yr – D)/ C = zM + (-A/C) → **zN = zM + K2**

Observație: atunci când un vector se suprapune pe suprafața unui poligon pot apărea defecte datorită modului diferit în care se calculează coordonatele z ale fragmentelor care se afișează în aceiași pixeli!

- numarul de comparatii de valori z pentru un pixel = nr de fragmente care se proiecteaza in acel pixel; timpul de calcul tinde sa devina independent de nr de poligoane: in medie, nr de pixeli acoperiti de un poligon e invers proportional cu numarul de poligoane
- “depth compleity” - nr suprascrieri ale unui pixel in bufferul de imagine, la generarea unui cadru imagine; calcul culoare fragment: model iluminare, utilizare texturi
- pentru o precizie buna e necesara repr pe 32 de biti a valorilor din z-buffer

- **ALGORITMUL BSP - BINARY SPACE PARTITIONING**
  - folosit initial doar pentru operatia de eliminare a obiectelor nevizibile din banda grafica (object culling)
  - a devenit alg pentru **reprezentarea si afisarea scenelor 3d cu eliminarea partilor nevizibile**
  - eficient pt hidden surface removal pt poligoane, dar poate fi folosit si pentru eliminarea partilor nevizibile ale liniilor dintr-o scena compusa din poligoane si linii
  - intrarea : lista primitivelor care compun scena 3d indiferent de ce obiect apartin
  - scena 3d reprezentata printr-un arbore binar : arbore BSP
  - arborele BSP e independent de pozitia observatorului - repr scena in sistemul coord globale
- **construirea arborelui BSP al unei scene**
  - fiecare nod coresp unui plan de partitionare al spatiului 3d(planul unui poligon al scenei)
  - fiecare plan de partitionare imparte spatiul in 2 semi-spatii
    - cel din fata planului(de aceeasi parte cu normala la plan)
    - cel din spatele planului



1. se incepe cu un poligon oarecare din lista (primul) pentru care se creeaza nodul radacina al arborelui
   a. primitivele din semi-spatiul fata formeaza lista fata care genereaza subarborele fata al nodului
   b. cele din spate formeaza lista spate care genereaza subarborele spate al nodului
2. se alege un poligon din lista fata si se creeaza nodul radacina al subarborelui fata si tot asa

- **afisarea arborelui BSP al unei scene:**
  - **tine cont de pozitia observatorului**
  - **afisare "front-to-back" si "back-to-front"**
- **afisare back-to-front**
  - se incepe cu primitiva cea mai indepartata de observator
  - se porneste din radacina arborelui si se avanseaza in arbore pana la frunze, in functie de poz observatorului fata de planul atasat fiecarui nod
  - in fiecare nod : se avanseaza in subarborele nodului care e opus observatorului fata de plan
- **eliminarea obiectelor aflate in afara volumului vizual(frustum culling)**
  - daca planul de partitionare al unui nod nu intersecteaza volumul vizual, atunci numai subarborele aflat de aceeasi parte cu volumul vizual va fi afisat, celalalt subarbore fiind eliminat din banda grafica. La fel si primitivele atasate nodului
- arborele nu trebuie sa fie construit pentru fiecare frame : avantaj pentru scenele statice
- modificarea pozitiei obs -> executia functiei de afisare daca scena nu s a modificat
- permite eliminarea din banda grafica a unui nr mare de poligoane din afara volumului vizual (obj culling)
- pt obiectele dinamice se pot face adaptari:
  - pt fiecare cadru se insereaza in arborele BSP cu obiecte statice, noduri coresp obiectelor dinamice
  - se transmite arborele static BSP nemodificat si obiectele dinamice vizibile, eliminarea partilor obturate din acestea fiind realizata in alg z-buffer
- **daca primitivele grafice sunt trimise in banda grafica in ordinea back-to-front, nu mai este necesat testul de adancime la rasterizarea primitivelor**
  - testul poate fi activat/.dezactivat cu glEnable(GL_DEPTH_TEST) / glDisable(GL_DEPTH_TEST)
- daca se foloseste BSP back-to-front impreuna cu testul de adancime, nr de suprascrieri ale pixelilor este mai mare decat in cazul unei ordini aleatorii a polig
- daca se foloseste BSP front-to-back cu test de adancime, afisarea e mai rapida decat folosind BSP back-to-front fara test, in functie de amplasarea polig
- **Construirea și afișarea arborelui BSP sunt funcții de management al scenei 3D, implementate de regulă într-un motor grafic 3D**

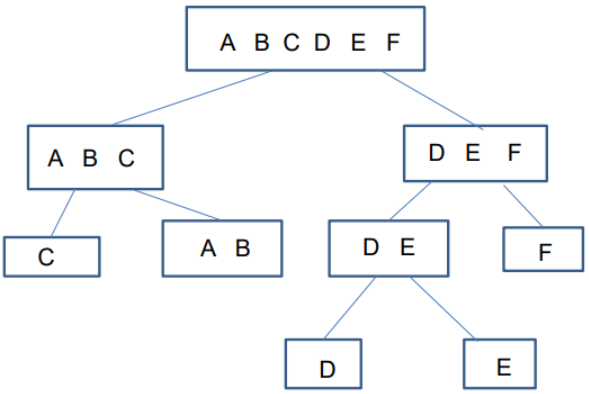
- **ALGORITMUL PICTORULUI**
    - “hidden surface removal”
    - intrare : lista poligoanelor care alcătuiesc scena 3D, transformate în spațiul de afișare
    - **eliminarea părților nevizibile se rezolvă prin afișare “din spate în față”, la fel ca în pictură.**
    - **nu e necesar testul de adancime la rasterizare**
    - **FORMA GENERALA:**
        1. Se calculează “extensia” fiecărui poligon din listă pe axele OX, OY, OZ (xmin,ymin,zmin – xmax, ymax, zmax): paralelipipedul încadrator al poligonului, cu fețele paralele cu planele principale ale sistemului de coordonate.
        2. Se ordonează poligoanele crescător după coordonata zmin a fiecărui poligon: **primul în listă va fi cel mai apropiat de observator.**
        3. Se descompun poligoanele ale căror extensii pe axa OZ se suprapun, astfel încât extensiile lor pe axa OZ sa fie disjuncte
        4. Se transmit în banda grafică poligoanele începând cu ultimul din listă (cel mai îndepărtat de observator).
    - sunt multe aplicatii in care nu e necesara descompunerea polig ale caror extensii pe OZ se suprapun, poligoanele fiind amplasate in plane de Z-constant : cartografie, generarea straturilor circuitelor imprimate, etc
    - nu e necesara descompunerea daca:
        - extensiile pe OX/ OY nu se suprapun
        - proiectiile poligoanelor nu se suprapun in imagine
    - testele se efectueaza progresiv in functie de complexitatea calculelor presupuse
    - principalul efort de calcul : **sortarea listei de poligoane si descompunerea, daca e necesara**

CURS 8
ELIMINAREA PARTILOR NEVIZIBILE ALE SCENELOR 3D DIN IMAGINI- STRUCTURI DE DATE ACCELERATE

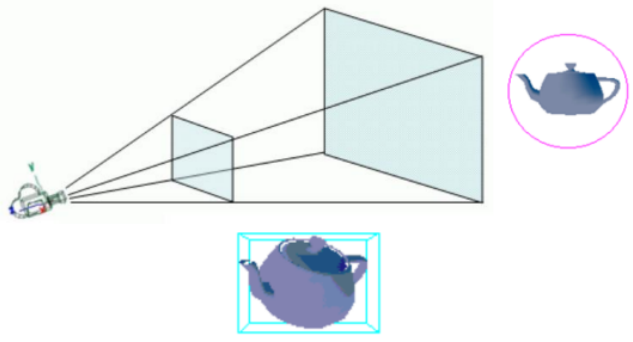
- **STRUCTURI DE DATE SPECIALE PENTRU MANAGEMENTUL SCENEI 3D**
    - ierarhie de volume incadratoare(BVH)
    - arbore BSP
    - arbore AA-BSP
    - arbore octal
- managementul scenei e realizat in cadrul unui motor grafic 3D, implementat peste OpenGL/Direct 3D
- **BOUNDING VOLUME HIERARCHY - BVH**
    - scena = arbore in care nodurile sunt volume încadratoare de obiecte sau grupuri de obiecte: sfera, paralelipipedul aliniat cu axele (Axis-Aligned

minimum Bounding Box: AABB), poliedrul convex minimal încadrator, cilindrul, elipsoidul.

- construirea arborelui:
  - top-down - radacina arborelui e asocaita cu intreaga scena
    - Se partiționează setul de obiecte în 2 sau mai multe subseturi și se alocă fiecare subset unui volum încadrator – rezultă primul nivel al arborelui
    - Se continuă procedura recursiv la nivelul fiecărui grup de pe nivelul anterior până când se ajunge la un singur obiect (sau grup de obiecte) pe nod – nod frunză
    - nu produce in general arbori optimi
  - bottom-up
    - se pleaca de la setul de obiecte si se defineste un volum incadrator pentru fiecare obiect/grup de obiecte - nodurile frunza
    - se grupeaza 2 sau mai multe volume nod frunza -> alte noduri ale arborelui
    - se continua gruparea pana cand totul e grupat intr-un singur volum -> radacina



  - bottom-up produce arbori mai eficienti decat cea top-down
- afisarea scenei 3d folosind arborele BVH
  - daca volumul incadrator al unui nod este in afara volumului vizual,
  - atunci obiectele din subarborele sau nu sunt trimise in banda grafica



        - testele de vizibilitate incep cu radacina arborelui

dacă volumul încadrator al nodului curent este complet în afara volumului vizual

atunci se revine pe nivelul anterior al arborelui (întregul grup de obiecte din subarborele nodului curent este exclus din banda grafică)

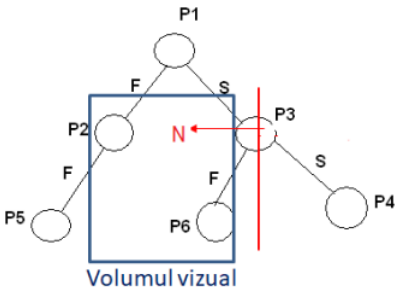
altfel se coboară în arbore;

daca se ajunge la o frunză al cărei volum încadrator nu este în afara volumului vizual,

atunci - grupul de primitive (obiectul) asociat frunzei este trimis în banda grafică;

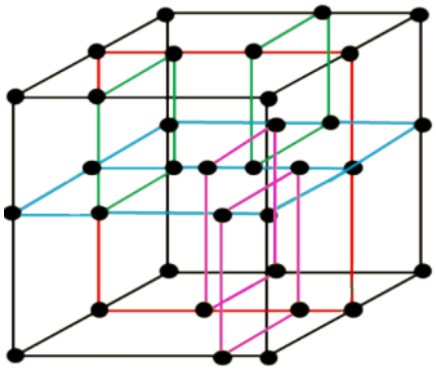
- se revine pe nivelul anterior al arborelui;

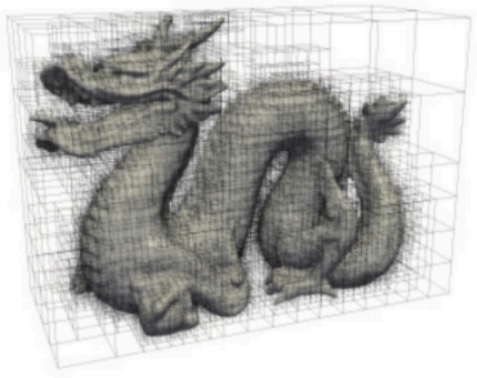
        - pt simplitate si eficienta, testele de vizibilitate se pot face in spatiul coord de decupare, transformand volumele incadratoare din matricea MVP si testand intersectia cu volumul vizual canonic
        - frecvent utilizata pentru detectia coliziunilor intre obiectele unei scene dinamice
    - **eficienta structurii BVH privind eliminarea obiectelor nevizibile din banda grafica**
        - depinde de cum se grupeaza obiectele in volume incadratoare - eficienta pentru unele pozitii ale obs, ineficienta pt altele
        - alegerea tipului de volum incadrator pt nod:
            - sa aibe forma simpla pt ca testele de intersectie cu volumul vizual sa fie simple si rapide
            - sa incadreze minimal obiectul
        - proprietatile dorite ale ierarhiei de volume incadratoare:
            - volumele nodurilor dintr-un subarbore trb sa fie apropiate spatial, cu atat mai apropiate cu cat sunt situate mai jos in arbore
            - volumul fiecarui nod sa incadreze minimal obiectul/grupul
            - volumul incadrator al scenei sa fie minimal
            - suprapunerea volumelor nodurilor vecine sa fie minimala
- **ARBORE BSP (POLYGON ALIGNED BSP TREE)**
    - arborele se construieste ca in alg BSP
    - **eliminarea grupurilor de primitive nevizibile:**
        - daca planul de divizare atasat unui nod nu intersecteaza volumul vizual, atunci subarborele atasat nodului situat in partea opusa planului fata de volumul vizual este complet nevizibil si poate fi eliminat din procesul de redare (P3,P4)(P5)(P1)

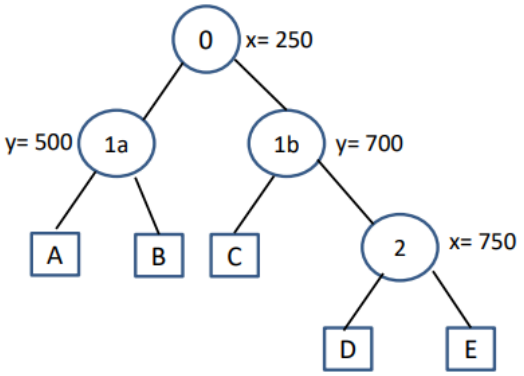


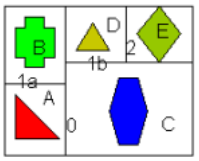
        - la fel si primitivele atasate nodului
        - eliminarea - in spatiul de coord globale, folosind volumul vizual defifnit de aplicatie (trunchi de piramida / paralelipiped dreptunghic)

- **AXIS ALIGNED BSP TREE (AA-BSP)**
  - **Planele de partiționare** sunt aliniate cu axele sistemului de coordonate 3D - **perpendiculare pe cele 3 axe**: Axis-Aligned BSP Tree (AA-BSP)
  - este un k-d tree : arbore binar in care fiecare nod corespunde unei coordonate intr-un spatiu k-dimensional;
  - k = 3 -> un nod va coresp uneia din coord x,y sau z
  - dacă planul de partiționare al unui nod este perpendicular pe axa OX, atunci nodul este asociat coordonatei x a planului.
  - **construirea arborelui:**
    - scena e divizata recursiv prin plane paralele cu planele sist de coord., intr-o ordine fixa, tinand cont si de geometria scenei
    - ex. în pasul 1 se alege un plan perpendicular pe OX, în pasul 2 un plan perpendicular pe OY, în pasul 3 un plan perpendicular pe OZ, în pasul 4 un plan perpendicular pe OX, s.a.m.d
  - **alegerea planelor:**
    - nr de obiecte din fiecare semispatiu sa fie acelasi, sau sa fie diferit cu 1, pentru ca arborele sa fie echilibrat
    - numarul de intersectii dintre planele de partitionare si obiectele scenei sa fie minimizat





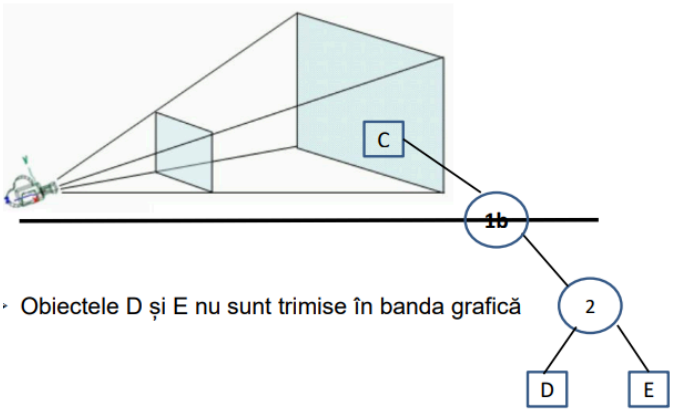




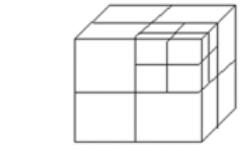
Exemplu de divizare
cu plane aliniate cu axele, în 2D:

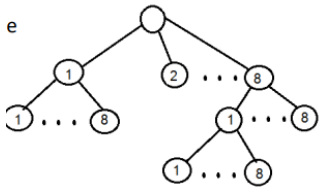
În fiecare pas planul de divizare este ales a.i. să se evite intersecția sa cu obiectele scenei.

- **eliminarea obiectelor nevizibile** : la fel ca la PA-BSP (: dacă planul de partiționare al unui nod nu intersectează volumul vizual, întregul grup de obiecte din subarborele aflat de partea opusă volumului este eliminat din banda grafică.)
- **afisarea scenei 3d folosind arborele AABSP**
    - Dacă planul atașat unui nod nu intersectează volumul vizual atunci întregul grup de obiecte din subarborele aflat de partea opusa volumului NU este trimis în banda grafică



- **ARBORE OCTAL**
    - **construire**
        - Se pleacă de la volumul încadrator al scenei, aliniat cu axele, care se asociază nodului rădăcină al arborelui.
        - Se divizează volumul încadrator în 8 sub volume egale
        - Se împart primitivele între cei 8 octanți
        - Se continuă cu divizarea recursivă a sub volumelor și asocierea primitivelor cu sub volumele; **fiecare primitivă este asociată cu cel mai mic octant care o încadrează complet.**
        - Un subvolum **nu este divizat dacă s-a ajuns la un nivel de divizare prestabilit** sau sub volumul conține suficient de puține primitive sau nici o primitivă
        - Rezultă un arbore in care fiecare nod intern (care nu este frunză) are 8 copii.

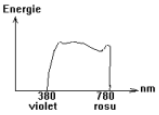




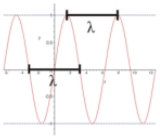
    - **eficienta:**
        - - **Subdivizarea strictă** a spațiului împreună cu **metoda de asociere a fiecărei primitive cu cel mai mic octant care o încadrează complet**, poate conduce la **ineficiență**; de ex., daca o primitivă este situată în apropierea centrului volumului încadrator al scenei, primitiva va fi asociata cu acest volum, chiar daca este foarte mică în raport cu volumul!
        - o alternativa: : o primitivă poate fi asociată cu 2 sau mai mulți octanți vecini care împreună o încadrează complet și formează un volum minimal (crește însă complexitatea construirii arborelui).

CURS 9
PROPRIETATILE LUMINII - MODELE DE CULOARE

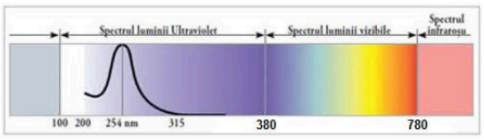
- **LUMINA**



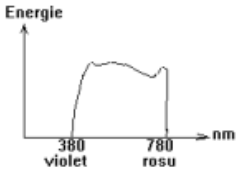
Spectrul vizibil



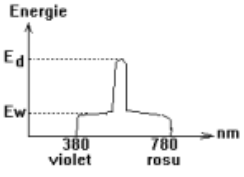
Lungimea de undă: distanța dintre două puncte din spațiu între care unda a parcurs o perioada de repetiție.



- **PROPRIETATILE LUMINII**
  - poate fi absorbita, reflectata sau transmisa
  - sistemul vizual uman percepe
    - lumina provenita direct de la o sursa
    - lumina reflectata/ transmisa de obiectele din mediu
  - **lumina acromatica =** are toate lungimile de unda din spectrul vizibil in proportii aprox. egale
    - daca e provenita de la o sursa e perceputa ca alba
    - Obiectele care reflectă acromatic mai mult de 80% din lumina incidentă albă apar ca albe. Cele care reflectă acromatic mai puțin de 3% din lumina incidentă apar ca negre.
  - **lumina cromatica =** predomina anumite lungimi de unda
    - provenita de la un obiect e perceputa ca alba, neagra sau ca nuanta de gri in functie de prop. fizice ale suprafetei obiectului
  - energia electromagnetica nu are culoare
  - Culoarea unui obiect, percepută de ochiul uman, depinde atât de distribuția lungimilor de undă în lumina care cade pe obiect cât şi de caracteristicile fizice ale suprafeței obiectului: reflectă/absoarbe anumite lungimi de undă
  - **culoarea este rezultatul unui proces psiho-fiziologic**
  - **definitia psiho-fiziologica a unei culori**
    - **nuanta** - determinata de lungimea de unda dominanta (rosu, verde ..)
    - **luminozitate (shine) -** intensitatea luminii, caracteristica unui obiect emitator de lumina
    - **saturatie -** masura a amestecului de alb intr-o culoare pura
      - caracterizeaza un obiect ne emitator, care reflecta lumina
      - o culoare pura are saturatia 100%
      - lumina acromatica are saturatia 0%
      - permite sa se faca distinctia intre nuante



Lumina acromatica



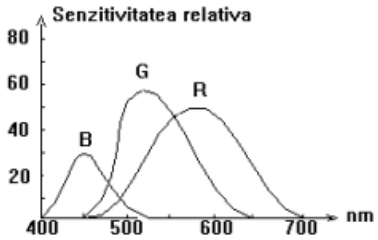
Lumina cromatica

Culoarea este data de lungimea de unda dominantă.

Ew=0 – culoarea are puritatea 100% : monocromatică

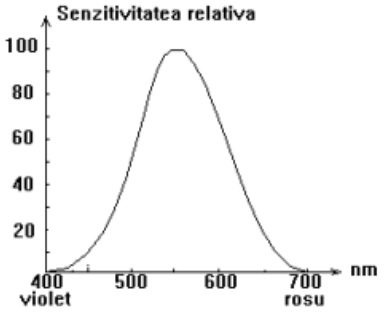
(lărgime de banda de un nanometru)

- **TEORIA TRICROMATICITATII**
  - **in retina ochiului uman exista 3 tipuri de conuri, fiecare fiind sensibil la una din culorile rosu, verde, albastru(culori primare)**
  - orice culoare care poate fi perceputa de ochiul uman se obtine printr-un amestec aditiv al celor 3 culori primare
  - prin amestecul lor in proportii egale -> alb



Raspunsul ochiului la cele 3 culori primare

Senzitivitatea maximă corespunde luminii cu lungimea de undă în jur de 550 nm, percepută ca galben-verde.



Curba de luminozitate

Suma celor 3 curbe de senzitivitate

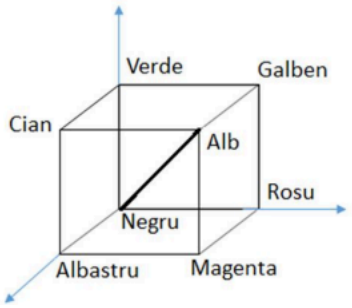
R G B

C M Y

  - 2 culori care prin amestec **produc lumina alba** se numesc **complementare**
  - culorile complementare culorilor primare : Cyan, Magenta, Yellow
  - Amestecând două culori primare în proporții egale se obține culoarea complementară celei de a treia
  - albastru+verde=cian, roşu+verde=galben, roşu+albastru= magenta
  - **R,G,B** -> **primitive aditive** - permit formarea de culori prin adunarea lor in diferite proportii
  - acest mod de definire al culorilor coresp. echipamentelor emitatoare de lumina(disp de vizualizare cu ecran)
  - **C, M, Y** se numesc **"primitive extractive**".(se obtin prin extraferea culorilor primare din lumina alba -> cyan = alb - rosu, magenta = alb -verde, galben = alb - albastru
    - **folosite pt a defini culorile reflectate de documente imprimate:**
      - pigmenții existenți în cernelurile tipografice absorb din lumina incidentă componentele complementare acelora ale pigmenților.
      - pigmentul de culoare magenta absoarbe din lumina incidentă lumina verde, iar cel de culoare galben, lumina albastra.
    - O suprafață care conține pigmenți magenta și galben va reflecta (sau transmite) lumină roșie.
    - permit **specificarea de nuante prin extragerea culorilor primare in diferite proportii din alb; scazandu-le in proportii efale din alb se obtin diferite nuante de gri**
- **MODELE DE CULOARE FOLOSITE IN SINTEZA IMAGINILOR**
  - **Orientate catre echipamente**
    - bazate pe culorile primare folosite de echipamente : RGN, CMY
    - YIQ(Luminance, In-phase, Quadrature) : pt transmiterea imaginilor TV color

- **Orientate catre utilizator**
  - bazate pe proprietatile psiho-fiziologice ale culorilor: HSV si HSL
- Un model de culoare specifică un sistem de coordonate 3D și un subspațiu al culorilor în sistemul de coordonate respectiv.
- Fiecare culoare se reprezintă printr-un punct în subspațiul culorilor.

Modelul RGB (Red, Green, Blue)



Roșu: 1,0,0

Verde: 0,1,0

Albastru: 0,0,1

Alb: 1,1,1; Negru: 0,0,0

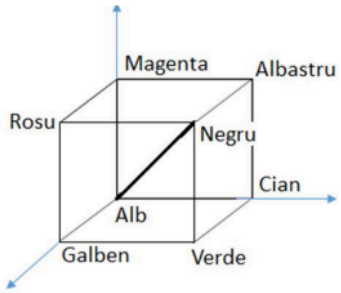
Cian: 0, 1, 1 - culoarea complementară culorii Roșu !

Pe diagonala Negru-Alb: nuanțe de gri – obținute prin amestec în proporții egale între R,G,B

Modelul CMY (Cyan, Magenta, Yellow)



CMYK (Cyan, Magenta, Yellow, Key) → key = negru



Cian: 1,0,0

Magenta: 0, 1, 0

Galben: 0,0,1

Alb: 0,0,0

Negru: 1,1,1 - (R,G,B) sunt extrase în proporție maximă din lumina alba

Roșu: 0, 1, 1 - Complementara culorii Cian!

Unele dispozitive de imprimare includ culoarea negru între culorile C,M,Y: CMYK

Conversia RGB ←→ CMY

[C M Y] = [1 1 1] - [R G B]
[R G B] = [1 1 1] - [C M Y]

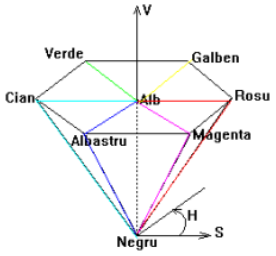
Culorile reflectate (RGB):
Cian: 1,1,1 – 1,0,0 = 0,1,1
Magenta: 1,1,1 – 0,1,0 = 1,0,1
Negru: 1,1,1 – 1,1,1 = 0,0,0

Modelul HSV (Hue, Saturation, Value)

- Model de culoare orientat către utilizator
- Artiştii specifică culorile prin tente, nuanțe şi tonuri



0<= V<= 1, 0<= S <= 1, 0<= H <= 360



V=1 - baza hexaconului

S=0 - pe axa hexaconului

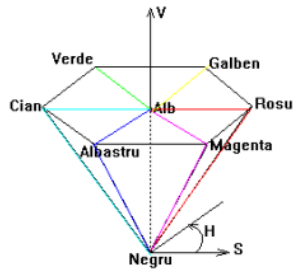
S=1 - pe frontiera bazei hexaconului: culorile pure

Roșu: (0, 1, 1)

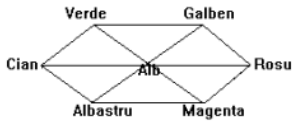
Culorile complementare – la 180 grade pe baza hexaconului

Cian: (180, 1, 1)

- Hexaconul din figură reprezintă subspațiul culorilor în modelul HSV



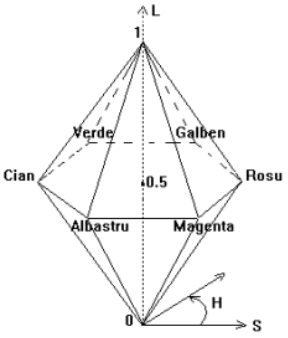
- Saturația într-un punct este definită prin distanta de la axa hexaconului la punctul respectiv.
- Combinația (S=0,V=1) corespunde culorii alb iar prin (S=0, 0<=V<=1) se reprezintă nivelurile de gri.
- Pentru S=0 valoarea lui H este nesemnificativă.
- Culorile pure sunt reprezentate prin V=1 și S=1.



- Baza hexaconului corespunde suprafeței văzute atunci când se privește cubul RGB dinspre vârful (1,1,1) de-a lungul diagonalei principale.

- Fiecare plan de V constant din spaţiul HSV corespunde vederii unui sub cub din cubul RGB.
- Diagonala principală a cubului RGB devine axa principală în spațiul HSV.
- Această interpretare sta la baza algoritmilor de conversie între modelele RGB şi HSV.

Modelul HSL (Hue, Saturation, Lightness)



Modelul HLS

Culorile primare cu saturație maximă şi complementarele lor sunt reprezentate prin S=1, L=0.5.

Conversii RGB → HSV/ HSL

max = sup(R, G, B) min = inf(R, G, B)

V = max

S= (max-min)/max daca max ≠ 0
0 daca max =0

H = 0 daca (max – min) = 0
(G – B) / (max – min) daca R = max
(B – R) / (max – min) + 2 daca G = max
(R – G) / (max – min) + 4 daca B = max
H = H x 60
daca H<0 H = H + 360

RGB → HSV

L = (max-min)/2
S = (max – min)/(max + min) daca L<=0.5
= (max – min)/ (2 – max – min) daca L > 0.5

H = 0 daca (max – min) = 0
(G – B) / (max – min) daca R = max
(B – R) / (max – min) + 2 daca G = max
(R – G) / (max – min) + 4 daca B = max
H = H x 60
daca H<0 H = H + 360

RGB → HSL

- **INTERPOLAREA IN SPATIUL CULORILOR**
  - Rezultatul interpolării între două culori **depinde de modelul de culoare** în care sunt specificate.

Fie două culori specificate în modelul RGB, C1=(1,0,0) şi C2=(0,1,0). Le interpolăm cu ponderi egale în modelele RGB și HSV:

Interpolare in modelul RGB:

C=t*C2 + (1-t)*C1 unde t=0.5, se obține

C=(0.5, 0.5, 0), care convertită în HSV ne dă (60, 1, 0.5)

Interpolare in modelul HSV:

C1 se reprezintă în HSV prin (0, 1, 1) iar

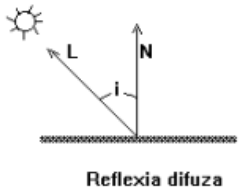
C2 prin (120, 1, 1)

C= 0.5*(0,1,1) + 0.5*(120,1,1) = (60, 1, 1)

  -

CURS 10
MODELE DE REFLEXIE

- **MODELE EMPIRICE PENTRU CALCULUL REFLEXIEI LUMINII**
  - **caracteristicile luminii reflectate intr-un punct al suprafetei depind de:**
    - lungimile de unda continute in lumina incidenta
    - proprietatile materialului suprafetei
    - directia si geometria sursei luminii incidente
    - orientarea suprafetei in pct respectiv
  - **expresia care modeleaza intensitatea luminii reflectate** intr-un pct al suprafetei este **definita pentru o suprafata necolorata si o lumina monocromatica** (caracterizata printr-o anumita lungime de unda lambda)
  - **in cazul general, lumina reflectata nu este monocromatica** -> expresia ar trebui sa fie evaluata continuu pe intregul domeniu al spectrului de modelat
  - În practică, expresia se evaluează separat pentru cele trei culori primare, R, G, B.
- **Aproximarea reflexiei difuze intr-un punct al unei suprafete 3D**
  - Lumina reflectată difuz de o suprafață este dispersată regulat în toate direcțiile.
  - **legea lui Larambert :** defineste e reflexia luminii provenite de la o **sursă punctiformă**, de către un **difuzor perfect:**



Reflexia difuza

Id=Isursa * kd * cos(i) 0<= i <= π/2

N – normala la suprafață în punctul considerat

Isursa – este intensitatea luminii incidente (provenită de la sursa de lumină)

Kd – este coeficientul de difuzie a luminii incidente, dependent de materialul suprafeței: suprafața reflectă/absoarbe/transmite lumina în anumite proporții: 0<=kd<=1

Id este intensitatea luminii reflectate difuz de suprafață

- **daca i > pi/2, suprafata nu primeste lumina de la sursa (sursa e in spatele suprafetei)**

- **Modelarea reflexiei luminii ambientale**
  - **lumina ambientala**
    - modeleaza lumina provenita de la celelalte obiecte ale scenei
    - este o sursa de lumina distribuita uniform in spatiu
    - **este reflectata la fel in orice punct al unei suprafete**
    - intensitatea reflexiei ei depinde de intensitatea luminii ambientale si proprietatile suprafetei
    - **Reflexia luminii ambientale este modelată prin:**
      - **Ia * ka**
        - Ia - intensitatea luminii ambientale
        - 0 <= ka <=1 coeficient de reflexie lumina ambientala, dependent de materialul suprafetei
      - **Id – intensitatea luminii reflectate difuz într-un punct al unei suprafețe**
      - **Id = reflexia luminii ambientale + reflexia difuză a luminii de la sursa = Ia * ka + Isursa * kd* cos(i)**
      - **0 <= i <= π/2**
- **Modelarea atenuarii luminii de la o sursa**
  - **Sursă direcțională:** sursa aflată la **distanță foarte mare (infinit)** de scena 3D
    - Vectorul L este același în orice punct al suprafeței
    - Pentru 2 **suprafețe paralele**, cu **aceleași proprietăți de material**, rezultă **aceeași Id → dacă proiecțiile suprafețelor se suprapun în imagine, ele nu se vor distinge.**
  - **Sursă pozițională:** sursa poziționată la o **distanță specifică** față de scena 3D:
    - Intensitatea luminii **descrește proporțional cu inversul pătratului distanței** de la sursa de lumină la suprafață.
    - **Id = Ia* ka + fat * Isursa* kd* cos(i)**
    - **fat = 1/d2** este **funcția de atenuare a luminii** de la o sursă d este distanța de la sursă la punctul de pe suprafață considerat.
    - Corecția nu satisface cazurile în care sursa este foarte îndepărtată: atenuare prea mare
    - distanta foarte mica -> atenuarile obtinute pt doua suprafete la distante putin diferite de sursa vor vi mult diferite

        - in practica : : fat = min(1/(c1 + c2*d + c3*d2), 1) c1, c2 şi c3 sunt trei constante care se asociază sursei de lumină
            - c1 se alege ai numitorul sa nu devina prea mic atunci cand sursa e foarte apropiata; valoarea functiei e limitata la 1 pentru a se asigura atenuarea
- **Culoarea luminii reflectate difuz**
    - Lumina incidentă poate conține mai multe lungimi de undă: → reflectate în mod diferit de o suprafață
    - Csursa = [IsursaR, IsursaG, IsursaB] – culoarea luminii sursei
    - Ca = [IaR, IaG, IaB] – culoarea luminii ambientale
    - **O suprafață reflectă în mod diferit componentele R, G, B ale luminii incidente.**
    - **ex. o suprafata care absoarbe din lumina incidenta componente G, B**
        - lumina incidenta e alba -> suprafata se vede rosie
            - [1,1,1] – [0,1,1] = [1,0,0]
        - lumina incidenta e rosie -> suprafata se vede rosie
            - [1,0,0] – [0,1,1] = [1,0,0] (g=0, b=0 ->nu pot fi extrase)
        - lumina incidenta e verde ->suprafata e vazuta ca neagra
            - [0,1,0] - [0,1,1] = [0,0,0]
    - **Rezultă: coeficienții de difuzie a luminii de către o suprafață depind de componentele luminii incidente și de proprietățile suprafeței**

ka = [kaR, kaG, kaB] – coeficienții de difuzie a componentelor R, G, B din lumina ambientală

kd = [kdR, kdG, kdB] – coeficienții de difuzie a componentelor R, G, B din lumina provenita de la sursă

- Expresia luminii reflectate difuz se evaluează separat pentru cele 3 componente ale luminii incidente, R,G,B:

  $Id\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda * kd\lambda * \cos(i)$, unde λ: lungimea de undă

sau:

- Cd = [IdR, IdG, IdB] - culoarea luminii reflectate difuz

  Cd = [IaR *kaR + fat*IsursaR *kdR *cos(i),

  IaG *kaG + fat*IsursaG *kdG *cos(i),

  IaB *kaB + fat*IsursaB *kdB *cos(i)]

- ka (kaR, kaG, kaB) și kd (kdR, kdG, kdB) sunt caracteristici ale materialului suprafeței, numite: culoarea ambientală și culoarea difuză a materialului.

Modelarea reflexiei difuze

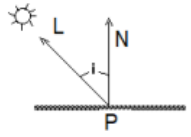
- Știind că

  cos(i)=L·N / (|L|·|N|) = Lu·Nu

  unde: N este normala la suprafață în P

  L este vectorul din P către sursa de lumină

  (L·N) – produsul scalar al vectorilor L, N



Reflexia difuza in punctul P

- Rezultă ecuația care modelează lumina reflectată difuz:

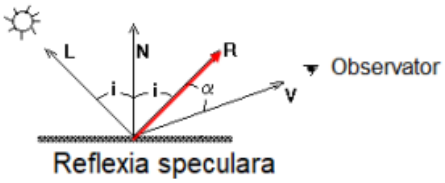
❖ Idλ = Iaλ*kaλ + fat*Isursaλ *kdλ* (Lu · Nu)

Cd = [IdR, IdG, IdB] – culoarea luminii reflectate difuz

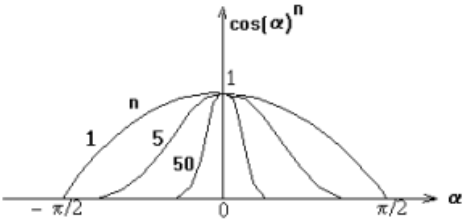
Pentru a include și cazul în care i> π/2 (lumina de la sursă nu ajunge în punctul considerat):

Iλ = Iaλ*kaλ + fat*Isursaλ *kdλ* max((Lu · Nu), 0)

- **Modelarea reflexiei speculare**
  - Un **reflector perfect**, de exemplu o oglindă, **reflectă lumina numai într-o singură direcție, R**, care este **simetrică cu L** față de normala la suprafață → numai un observator situat exact pe direcția respectivă va percepe raza reflectată:



Reflexia speculara



Functia de distributie spatiala a luminii reflectate specular

  - Pentru materialele **imperfect reflectante** cantitatea de lumină care ajunge la observator **depinde de distribuția spațială a luminii** reflectate specular:
    - suprafete netede - distributie dreapta, focalizata
    - suprafete cu rugozitati - dispersata
    - se aproximează prin cos(α)^n (modelul Bui-Tuong Phong) unde n este exponentul de strălucire (shininess) al materialului.

Modelul Phong pentru aproximarea reflexiei speculare într-un punct al unei suprafețe 3D:

Isλ = Isursaλ * w(i,λ) * cos(α)^n

w(i, λ) este funcția de reflectanță, i - unghiul de incidență iar λ - lungimea de undă a luminii incidente

În practică, w(i, λ) este înlocuit cu o constantă determinată experimental, numită **coeficientul de reflexie speculară al materialului, notat** ksλ.

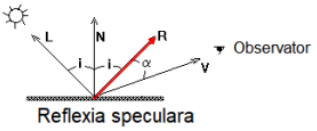
Modelul practic al reflexiei speculare:

Isλ = Isursaλ * ksλ * cos(α)^n

cos(α)=R · V / (|R|·|V|) = Ru·Vu

Rezultă:

Isλ = Isursaλ*fat*ksλ*(Ru · Vu)^n



Reflexia speculara

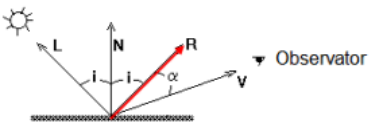
Pentru a include și cazul în care α >= 90:

Isλ = Isursaλ*fat*ksλ* max ((Ru · Vu)^n , 0)

Reflexia speculară nu poate avea loc dacă în punctul considerat nu se primește lumină de la sursă: $Is\lambda = lum * Isursa\lambda * fat * ks\lambda * \max((Ru \cdot Vu)^n, 0)$

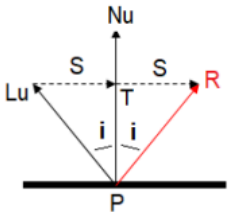
$lum = 1$ daca $(Lu \cdot Nu) > 0 \quad (0 <= i < \pi/2)$

$= 0$ altfel



Reflexia speculara

Calculul direcției reflexiei speculare



- Vectorul R este simetricul vectorului L față de N.
- Proiecția scalară a vectorului Lu pe direcția normalei este mărimea segmentului PT: $|Lu|\cos(i) = \cos(i) = Lu \bullet Nu$
- Proiecția vectorială a lui Lu pe direcția normalei este **vectorul PT**, care are lungimea segmentului PT și direcția normalei:

$(Lu \bullet Nu) * Nu / |Nu| = (Lu \bullet Nu) * Nu$

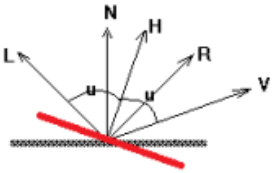
$R = Lu + 2 * S, \quad PT = Lu + S \rightarrow S = PT - Lu$

$R = Lu + 2(PT - Lu) = Lu + 2 * (Lu \bullet Nu) * Nu - 2 * Lu = 2 * (Lu \bullet Nu) * Nu - Lu$

$R = 2 * (Lu \bullet Nu) * Nu - Lu$ **sau în glsl: R = *reflect*(-Lu, Nu)** 😎

- **O altă formulare a modelului Phong pentru reflexia speculară folosește vectorul median, notat cu H în figură. El face unghiuri egale cu L și cu V.**
  - Dacă suprafața ar fi orientată astfel încât normala să aibă direcția lui H, atunci observatorul ar percepe lumina speculară maximă (deoarece ar fi pe direcția razei reflectate specular).
  - Atunci când sursa de lumină şi observatorul sunt la infinit, utilizarea termenului Nu•Hu este avantajoasă deoarece Hu este constant (același în toate punctele unei suprafețe).

razei reflectate specular).



❖ Termenul care exprimă reflexia speculară este în acest caz:

$Is\lambda = Isursa\lambda * fat * ks\lambda * (Nu \bullet Hu)^n$, unde $Hu = (Lu + Vu) / |(Lu + Vu)|$

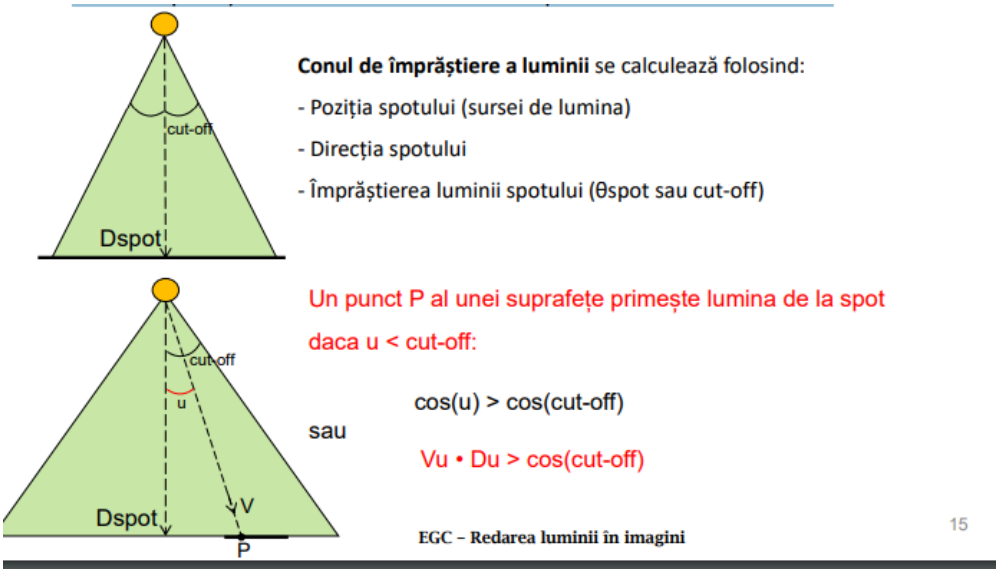
- **Modelul de iluminare locala (Model de reflexie Phong)**
  - combina culorile datorate:
    - reflexiei luminii ambientale – **culoarea_ambientala,**
    - reflexiei difuze a luminii de la o sursă / surse – **culoarea_difuza**
    - reflexiei speculare a luminii de la o sursă /surse– **culoarea_speculara**
    - **Reflexia luminii într-un punct al unei suprafețe determină culoarea sa în acel punct.**

❖ Dacă scena 3D este iluminată de m surse de lumină:

$$I\lambda = Ia\lambda * ka\lambda + \sum_{1<=i<=m} fat_i * Isursa\lambda_i * [kd\lambda * \max((Lu_i \bullet Nu), 0) + lum_i * ks\lambda * \max((Hu_i \bullet Nu)^n, 0)]$$

❖ La cele 3 componente se poate adăuga culoarea_emisa de suprafață. Rezultă:
Culoarea într-un punct al unei suprafețe
= culoarea_emisa + culoarea_ambientala + culoarea_difuza + culoarea_speculara

        - **culoarea emisa:**
            - lumina emisa de o suprafata
            - independenta de sursele de lumina
            - aceeasi in oricare punct al suprafetei
            - nu lumineaza obiectele din jur
            - culoarea_emisa = Ke, o constanta
- **Efectul de spot**
    - **În modelul de iluminare locală, sursa de lumină este considerată punctiformă. Lumina produsă de o sursa punctiformă este omnidirecțională.**
    - simulează o sursă de lumină care produce un con de lumină.
    - Numai suprafețele aflate în conul de lumină primesc lumina de la sursă.



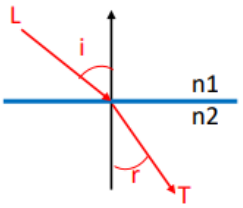
CURS 11
TRANSPARENTA IN MODELELE DE ILUMINARE LOCALA

- **MODELAREA TRANSPARENTEI**
    - unele obiecte din scena pot fi construite din materiale transparente/translucide
    - **transmisia luminii prin obiecte transparente : speculara**
    - **transmisia luminii prin obiecte translucide : difuza**
    - cand lumina trece dintr-un mediu in altul isi modifica directia datorita refractiei

      Relația dintre unghiul razei incidente, i, și cel al razei refractate, r, este dată de legea lui Snell:

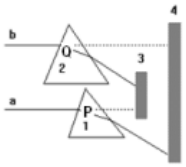
      sin(r)/sin(i) = n1/n2

      n1 și n2 sunt indicii de refracție ai celor două medii (materiale) traversate de lumină.

    -
    - **indicele de refractie dependent de lungimea de unda a luminii incidente**

- culoarea vizibila intr-un pct al uenei suprafete transparente provine de la obiectul aflat pe directia razei transmise



3 și 4 sunt poligoane opace

1 si 2 sunt poligoane transparente, aflate în fața poligoanelor 3 și 4

a și b sunt raze de lumină provenind de la surse, incidente în P și Q

Ținând cont de refracție:	Neglijând refracția (T are direcția razei incidente)
- Culoarea în P este dată de poligonul 4	- Culoarea în P este dată de poligonul 3
- Culoarea în Q este dată de poligonul 3	- Culoarea în Q este dată de poligonul 4

-
- Refracția produce, de asemenea, o distorsionare a obiectelor, asemănătoare cu aceea produsă de o proiecție perspectivă
- Multe **metode practice de modelare a transparenței ignoră refracția**, astfel încât **obiectele vizibile printr-o suprafață transparentă sunt cele aflate pe direcția razei incidente.**
  - Motivul ignorării:
    - reducerea volumului de calcule;
    - obținerea realismului fotografic în totalitate (fără deformare)
- când suprafața vizibilă într-un pixel este transparentă, culoarea în care va afișat pixelul se poate obține **combinând culoarea suprafeței vizibile** cu aceea a **suprafeței aflată imediat în spatele său (sau a fondului)**, folosind următoarea formulă de interpolare:

$I_\lambda = (1-kt_1)*I_{\lambda 1} + kt_1*I_{\lambda 2}$

$I_{\lambda 1}$ este culoarea suprafeței vizibile în pixel

$I_{\lambda 2}$ este culoarea suprafeței din spatele celei vizibile, care se proiectează în același pixel

Coeficientul de transmisie, kt_1, este o măsură a transparenței suprafeței vizibile în pixel,

$0 <= kt_1 <= 1$

$kt_1 = 0$ → suprafața vizibilă este opacă si deci pixelul va fi afișat în culoarea sa, $I_{\lambda 1}$

$k_{t1} = 1$ → suprafața vizibilă este perfect transparentă si nu contribuie la culoarea pixelului, pixelul va fi afișat în culoarea suprafeței din spate, $I_{\lambda 2}$

Dacă $k_{t1}=1$ și suprafața din spatele celei vizibile este la rândul său transparentă, metoda de calcul se aplică recursiv, până când se întâlnește o suprafață opacă sau fundalul scenei.

-
- Aproximarea liniară din model **nu dă rezultate bune pentru suprafețele curbe:**
  - în apropierea laturilor siluetei unei suprafețe curbe (de exemplu, o vază sau o sticlă) **grosimea materialului reduce transparența**
  - soluția propusă de D.S. Kay ("Transparency For Computer Synthesized Images"): **kt se calculează în funcție de normala la suprafață în punctul considerat**
- **Majoritatea algoritmilor de determinare a vizibilității suprafețelor la afișarea scenelor 3D pot fi adaptați pentru a îngloba transparența.**

- In algoritmii care afișează poligoanele scenei 3D în ordinea “din spate în față” (back to front), de ex. algoritmul Pictorului și BSP, Iλ1 corespunde poligonului care se rasterizează la un moment dat iar Iλ2 este valoarea existentă în memoria imagine pentru pixelul considerat.
- in z-buffer e mai dificil deoarece poligoanele sunt rasterizate in ordinea in care sunt transmise in banda grafica, indiferent de distanta lor fata de observator
  - Se poate realiza folosind mai multe memorii buffer-imagine și generând imaginea în mai multe etape, imaginea finală obținându-se prin combinarea imaginilor poligoanelor opace cu cele ale poligoanelor transparente
- În OpenGL, **transparența poate fi simulată** prin amestecul dintre culoarea fragmentului curent și cea a pixelului în care se afișează, folosind modelul **(R, G, B, A); A – opacitatea: 0 – transparent; 1 – opac**
  - Operația de amestec este o operație raster: efectuată de procesorul de rasterizare după testul de adâncime, înainte de actualizarea valorii pixelului în bufferul imagine (frame buffer)

glEnable(GL_BLEND) - activează amestecul culorilor

glBlendFunc(..) – specifică modul de amestec al culorilor. Exemplu:

glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA):

noua_culoare_pixel = Culoare_fragment * A_fragment + Culoare_pixel * (1 – A_fragment)

CURS 12
INTRODUCEREA CETII IN IMAGINI

- **MODELAREA CETII**
  - **Ceața adaugă realism imaginii**
  - **ajută în eliminarea defectelor produse de decupare**a la nivelul planului din spate al volumului vizual
    - Fără **ceață**, pe măsură ce observatorul se **îndepărtează de un obiect**, obiectul se apropie de planul din spate și este **decupat atunci când planul din spate îl intersectează**.
    - Cu ceață, **dacă intensitatea ceții crește proporțional cu distanța de la poziția observatorului**, se creează **efectul de distanță** pentru obiectele îndepărtate, iar o**biectele decupate la nivelul planului din spate nu mai sunt vizibile.**
  - Cfog - culoare ceata
  - Clocal - culoarea calculata intr-un pct folosind mdelul de iluminare locala
  - **0<= Φ <= 1 – factorul de ceață, proporțional cu distanța de la observator la punct**
  - **Culoarea punctului, afectată de ceață: Cloc_fog = (1-Φ)*Clocal + Φ*Cfog**
  - dist distanța de la observator la un punct din scena vizualizată.

Factorul de ceață liniar

- Acționează între 2 distanțe definite în sistemul coordonatelor observator, z0 si z1, măsurate pe direcția în care privește observatorul.



$\Phi = 0$ pentru dist < z0

$\Phi = (dist - z0) / (z1 - z0)$ pentru z0 <= dist <= z1

$\Phi = 1$ pentru dist > z1

Factorii de ceață exponențiali

$\Phi = 1.0 - e^{f}$

EXP: $f = -(fog_density*dist)$

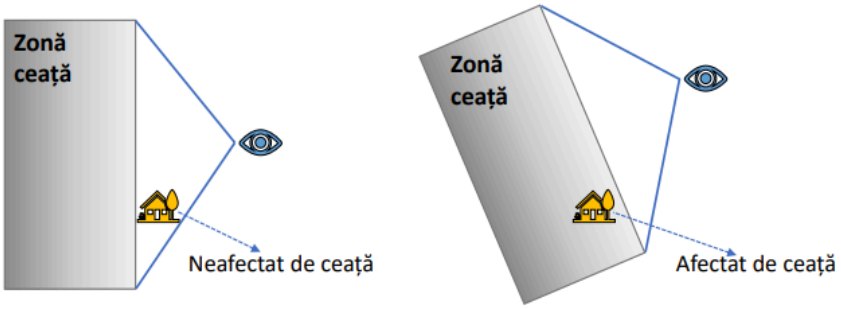
EXP2: $f = -(fog_density*dist)^2$

- fog_density: densitatea ceții (un număr real, de ex. 0.05) - modelează atenuarea luminii datorata ceții

3

1. **dist** este coordonata z a punctului, la fel ca distanțele z0 și z1

Dacă se rotește camera, fără a-i modifica poziția, coord. z a punctului se modifică → un obiect aflat la o distanță d de observator poate să intre/iasă din zona de ceață:



2. **dist** este lungimea vectorului de la observator (0,0,0) la punct.

- varianta 1 e mai ieftina computational si preferata atunci cand nu e rotita camera

- **INTRODUCEREA CETII IN IMAGINI**
  - **in vertex shader (modelul Gouraud)**
    - se calculează Φ folosind distanța (dist) de la observator la poziția vârfului transformată în coordonate observator
    - se calculează culoarea vârfului afectat de ceață: **Cloc_fog = (1-Φ)*Cvrf + Φ*Cfog**
    - **Cloc_fog este ieșire din vertex shader**
    - culorile afectate de ceață ale vârfurilor sunt interpolate de GPU la nivel de fragmente
    - culoarea de ceață interpolată, Cloc_fog, este intrare pentru fragment shader
  - **În fragment shader (modelul Phong):**
    - **Varianta (1): fragment shader primește poziția fragmentului în coordonate observator**
      - se transmite de la vertex shader poziția vârfului în coordonate observator

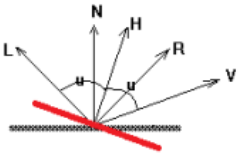
        - poziţiile vârfurilor în coordonate observator sunt interpolate de GPU la nivel de fragmente
        - în fragment shader se calculează Φ folosind ca distanța (dist) - poziția fragmentului (pe z) în coordonate observator
    - **Varianta (2) poziția fragmentului în coordonate observator se poate calcula în fragment shader** sau mai simplu, se poate primi (interpolata) de la VS

CURS 13
MODELE DE SHADING

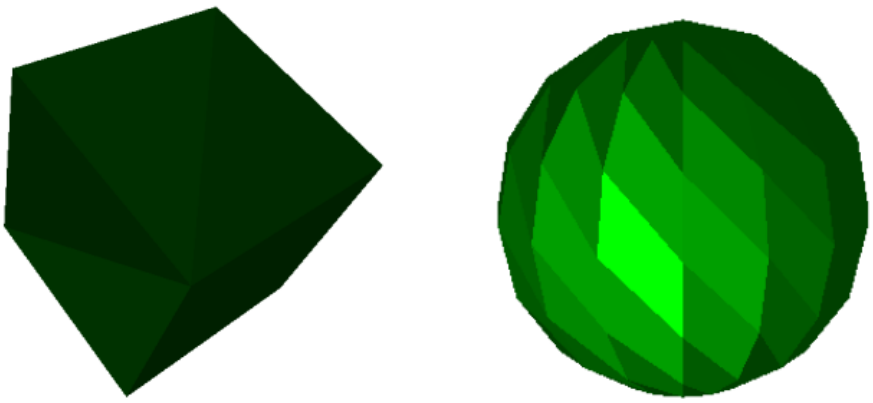
- **CALCULUL CULORII FRAGMENTELOR LA REDAREA SUPRAFETELOR 3D - SHADING MODELS**
- **MODELUL LAMBERT - ILUMINARE CONSTANTA SAU PLATA**

  Toate fragmentele unui poligon sunt afișate cu aceeași culoare, calculată folosind modelul de iluminare locală (de regulă considerând numai componenta difuză):

  $$Id\,\lambda = Ia\,\lambda * ka\lambda + fat * Isursa\,\lambda * [kd\lambda * \max((Nu \bullet Lu),0) + lum * \max((Nu \bullet Hu)^n,0)]$$

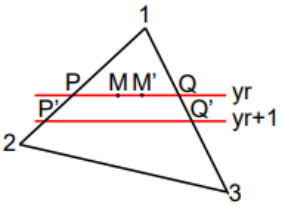
  

  -
  - **se bazeaza pe urmatoarele presupuneri:**
    - Poligonul face parte din suprafața de vizualizat și nu este o aproximare a unui petic de suprafață curbă: **vectorul Nu este același în orice punct al poligonului.**
    - Sursa de lumină este la infinit: **produsul scalar (Nu • Lu)** este atunci **constant pe întreaga suprafață** a poligonului.
    - Observatorul este la infinit: **(Nu • Hu) este constant pe suprafața poligonului;**
  - ultimele 2 nu sunt satisfacute -> conventie de calcul pentru vectorii V,L pentru un intreg poligon (ex. pot fi calculati in centrul poligonului)
  - prima nu e satisfacuta -> aproximarea unei suprafețe curbe prin rețeaua de fațete poligonale este vizibilă deoarece intensitățile calculate pentru fațete adiacente cu orientare diferită sunt diferite.



- **EFECTUL DE BANDA MACH**
  - **este o iluzie optică** datorată f**iltrării componentei de luminozitate** a imaginii formate pe retină, de către sistemul vizual uman
  - **mărește contrastul dintre zone adiacente cu intensitate ușor diferită**, la frontiera dintre zone, declanșând detecția frontierei în sistemul vizual uman.
  - Atunci când **o suprafață curbă** (de exemplu o sferă) este **descompusa în fațete poligonale**, este de dorit ca **laturile fațetelor să nu fie vizibile** în imaginea sferei.
  - in cazul folosirii modelului **Lambert** pentru calculul culorilor fragmentelor, **datorită folosirii unei singure normale pentru toată suprafața unei fațete**, **culorile fațetelor adiacente vor fi ușor diferite**, iar efectul de bandă Mach mărește percepția schimbării de intensitate pe laturile fațetelor adiacente
- **MODELUL GOURAUD**
  - La momentul rasterizării unui poligon, **fiecărui vârf al poligonului îi este asociată o culoare.**
  - **Culorile fragmentelor** poligonului se obțin prin **interpolarea liniară** a culorilor din vârfuri.
  - **culoarea** intr-un varf se calculeaza modelul de iluminare locala in varf (cu N,L,V in coord locale)
  - **normala** trebuie sa fie transmisa la vertex shader
    - poate fi calculata din ecuatia analitica a suprafetei pe care se afla varful
    - poate fi aproximata ca medie aritmetica a normalelor la fetele adiacente in varf
  - la rasterizarea unui poligon, **procesorul de rasterizare calculeaza culoarea fiecarui fragment astfel:**
    - prin interpolarea liniară a **culorilor vârfurilor**, pentru **fragmentele de pe laturi**
    - prin interpolare liniară între **culorile din capetele segmentului** interior **pe care se află fragmentul**.

Interpolarea liniară a culorilor



$y = y1 + t\,(y2 - y1) \rightarrow tP = (yr - y1)/(y2 - y1)$

$IP = I1 + tP\,(I2 - I1) = I1 + (I2 - I1)\,(yr-y1) / (y2-y1) = I1 + (yr-y1) * \mathbf{C_{1-2}}$

$\mathbf{C_{1-2} = (I2 - I1) / (y2 - y1)}$ **- constanta pentru latura 1-2**

$IQ = I1 + (I3 - I1)\,(yr-y1) / (y3-y1) = I1 + (yr - y1) * \mathbf{C_{1-3}}$

$\mathbf{C_{1-3} = (I3 - I1) / (y3 - y1)}$ **- constanta pentru latura 1-3**

$x = xP + t(xQ - xP) \rightarrow tM = (xM - xP)/(xQ - xP)$

$IM = IP + tM\,(IQ - IP) = IP + (IQ - IP)\,(xM - xP) / (xQ - xP) = IP + (xM-xP)* \mathbf{C_{P-Q}}$

$\mathbf{C_{P-Q} = (IQ - IP) / (xQ - xP)}$ **- constanta pentru segmentul P-Q**

Calculul incremental al culorilor

$IP' = I1 + (I2 - I1)\,(yr+1 - y1) / (y2 - y1) = I1 + (yr+1 - y1) * \mathbf{C_{1-2}} = IP + \mathbf{C_{1-2}}$

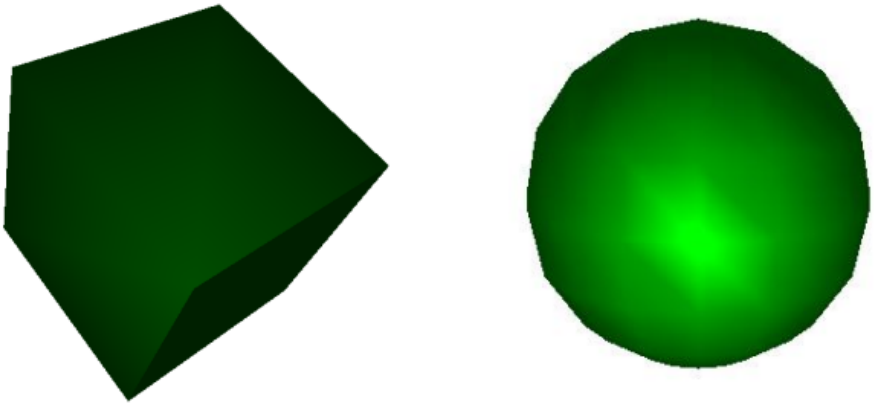
$\mathbf{IP' = IP + C_{1-2}}$

analog, $\mathbf{IQ' = IQ + C_{1-3}}$

M' (xM + 1, yr)

$IM' = IP + (xM + 1 - xP) * \mathbf{C_{P-Q}} \rightarrow \mathbf{IM' = IM + C_{P-Q}}$

- este integrat in alg de rasterizare a poligoanelor
- calcule simple pentru culoarea unui fragment (calcul incremental)->rapid
- **Nu permite calculul luminii reflectate specular pentru fragmentele interioare unui poligon (intensitățile fragmentelor interioare nu pot fi mai mari decât cele din vârfuri)**
- Reduce efectul de banda Mach, dar nu îl elimină:
  - Culorile pe latura de adiacență a 2 poligoane sunt aceleași
  - Nu ține cont de tangentele la fețele adiacente pe o latură (culorile de pe laturi se calculează folosind numai culorile din vârfuri)
  - Efectul de banda Mach poate fi observat în vecinătatea siluetei unei suprafețe 3D și a zonelor de curbura mare



- **MODELUL PHONG**
  - **la momentul rasterizarii unui pologon, fiecarui varf ii este asociata normala varfului,** transmisa prin programul de aplicatie
  - Normala și poziția vârfului, în coordonate globale, sunt ieșiri din vertex shader
  - Procesorul de rasterizare **interpolează liniar pozițiile și normalele vârfurilor**, **transmițând** la fragment shader **poziția și normala fragmentului**
  - in fragment shader **se calculează culoarea fragmentului aplicând modelul de iluminare locală în poziția fragmentului**
  - calculul normalei pentru fiecare fragment
    - prin **interpolarea liniara a normalelor vârfurilor**, pentru **fragmentele de pe laturi** (la fel ca în modelul Gouraud, pentru culori);
    - prin **interpolare liniară între normalele capetelor segmentului interior pe care se află fragmentul** (la fel ca în modelul Gouraud, pentru culori).

Dezavantaje față de modelul Gouraud

❑ Componentele Nx, Ny, Nz ale normalei unui fragment se obțin (în algoritmul de rasterizare) printr-un calcul incremental analog cu cel folosit pentru calculul culorilor în modelul Gouraud dar, pentru folosirea lor în calculul culorii, normala trebuie să fie normalizată în fragment shader:

Nu = N/|N|, unde |N| = $\sqrt{Nx^2 + Ny^2 + Nz^2}$ (se folosește funcția GLSL *normalize()*)

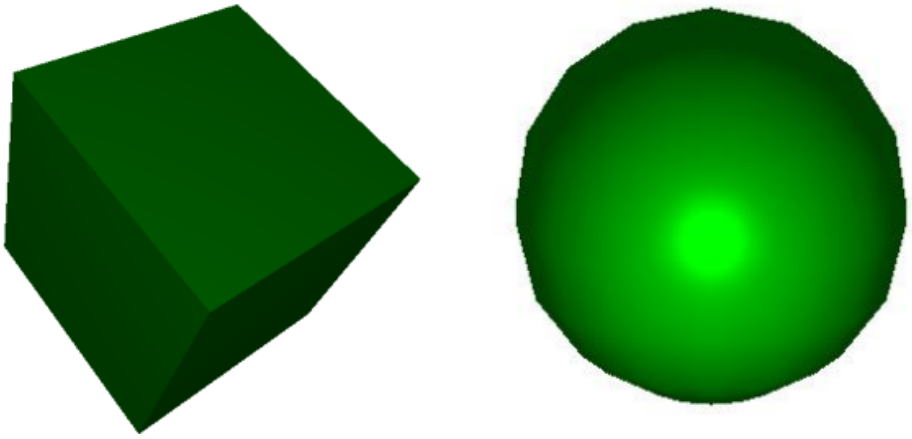
❑ De asemenea, vectorii L (către sursa de lumină) și V (către observator), folosiți în modelul de iluminare locală la nivel de fragment, sunt calculați și normalizați în programul fragment shader pentru a fi folosiți în calculul culorii fragmentului.

➢ Calcule mai complexe la nivel de fragment decât în modelul Gouraud

Avantaje

❑ Permite redarea reflexiei speculare in orice fragment al unui poligon

❑ Reduce mult efectul de bandă Mach



CURS 14
DETALII DE IMPLEMENTARE UMBRE

- **STENCIL BUFFER**
  - Stencil buffer-ul pune la dispoziție multe opțiuni pentru a restricționa desenarea pe ecran și se poate folosi în multe locuri acolo unde buffer-ul de adâncime nu poate fi utilizat.
  - Privit în modul cel mai simplist stencil buffer-ul este folosit pentru a bloca la desenare anumite zone din ecran.
  - Activare: glEnable(GL_STENCIL_TEST)
  - Exista 4 funcții stencil în OpenGL :
    - void glClearStencil(GLint s)
    - void glStencilFunc(GLenum func,GLint ref, GLuint mask)
    - void glStencilMask(GLuint mask)
    - void glStencilOp(GLenum fail, GLenum zfail, GLenum zpass)

- glClearStencil() funcționează în mod similar cu glClearColor și glClearDepth De asemenea inițializează stencil buffer-ul cu valoarea inițială dată ca parametru când este apelat glClear(GL_STENCIL_BIT). În mod default această valoare este 0.
- În mod default doar prin activarea folosirii stencil buffer-ului nu se întâmplă nimic toate desenările pe ecran fiind permise fără nici o restricție.
- Pentru a lucra cu stencil buffer-ul trebuie să "marcăm" valori în acesta folosind glStencilFunc() și glStencilOp()
- Funcția glStencilFunc(GLenum func, GLint ref, GLuint mask) definește : • o funcție de comparație •
  - o valoare de referință cu care se face comparația
  - • o mască (se face ȘI logic cu valoarea de referință && valoarea din stencil buffer)

Funcțiile de comparare valide pentru stencil test sunt :

- GL_NEVER – eșuează întotdeauna (nu se va desena nimic pe ecran)
- GL_ALWAYS – reușește întotdeauna (desenarea are loc mereu - aceasta este comportarea default)
- GL_EQUAL – reușește dacă valoarea de referință este egală cu valoarea din stencil buffer
- GL_NOTEQUAL – reușește dacă valoarea de referință nu este egală cu valoarea din stencil buffer
- GL_LESS – reușește dacă valoarea de referință este mai mică decât valoarea din stencil buffer
- GL_LEQUAL – reușește dacă valoarea de referință este mai mică sau egală cu valoarea din stencil buffer
- GL_GREATER – reușește dacă valoarea de referință este mai mare decât valoarea din stencil buffer
- GL_GEQUAL – reușește dacă valoarea de referință este mai mare sau egală cu valoarea din stencil buffer

Legate de funcțiile stencil sunt operațiile stencil definite cu
glStencilOp(GLenum fail, GLenum zfail, GLenum zpass) care setează comportarea valorii stencil bufferului pentru cei trei parametri ai funcției :

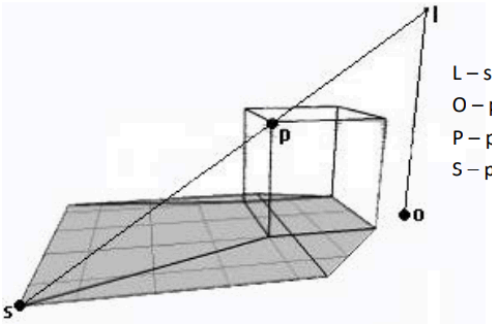
- fail : testul stencil a eșuat
- zfail : testul stencil a reușit dar testul de adâncime a eșuat
- zpass : testul stencil a reușit și testul de adâncime a reușit

Operațiile valide asupra valorilor din stencil buffer sunt :

- GL_ZERO - setează valoarea din stencil pentru pixelul/fragmentul curent la 0
- GL_KEEP - păstrează valoarea din stencil pentru pixelul/fragmentul curent
- GL_REPLACE – înlocuiește valoarea din stencil pentru pixelul/fragmentul curent cu valoarea de referință
- GL_INCR – mărește valoarea din stencil pentru pixelul/fragmentul curent
- GL_DECR - scade valoarea din stencil pentru pixelul/fragmentul curent

Exemple de folosire :

- glStencilFunc(GL_ALWAYS, 1, 1); glStencilOp(GL_KEEP, GL_KEEP, GL_REPLACE);
  Se va desena tot timpul pe ecran iar în locul unde a fost desenat ceva stencil buffer-ul va avea valoarea 1
- glStencilFunc(GL_EQUAL, 0, 1); glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);
  Se va desena pe ecran numai unde stencil buffer-ul este 0 și valorile din stencil buffer vor fi păstrate

- **UMBRE PLANARE**
  - permite afisarea de umbre pe suprafete plane
  - ideea: gasirea unei transformari care proiecteaza geometria fiecarui obiect generator de umbre pe un plan stabilit rezultand astfel umbra lui
  - proiectia poate fi descrisa de o matrice 4x4 de transformare in coord omogene
  - 

L – sursa de lumină punctiformă
O – punct de pe plan
P – punct al obiectului generator de umbră
S – proiecția lui P pe plan

Pentru a simplifica lucrurile să considerăm pentru început situația din figură unde avem o sursă de lumină la poziția L = (Lx,Ly,Lz) și planul care va primi umbra este y = 0

Un punct P = (Px,Py,Pz) de pe obiect văzut din sursa de lumina L va proiecta o umbră pe plan în punctul S = (Sx,0,Sz).

Punctul S poate fi exprimat ca:

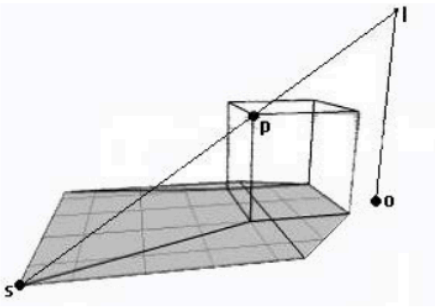
$$s = L + t(P - L) \qquad (1)$$

Dar cum Sy = 0 rezultă:

$$t = \frac{L_y}{L_y - P_y}$$

Introducând în formula (1) obținem coordonatele lui S:

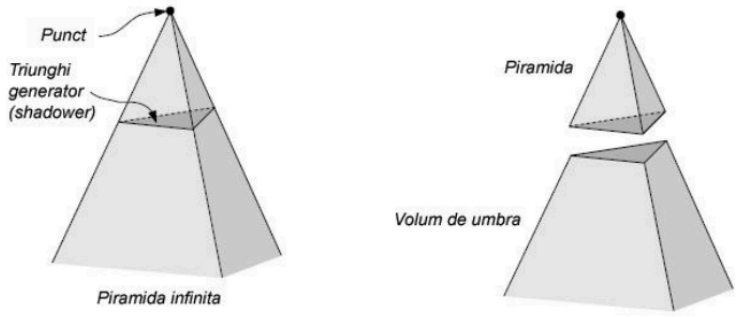
$$\begin{bmatrix} S_x \\ 0 \\ S_z \\ w \end{bmatrix} = \begin{bmatrix} L_y & -L_x & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -L_z & L_y & 0 \\ 0 & -1 & 0 & L_y \end{bmatrix} \begin{bmatrix} P_x \\ P_y \\ P_z \\ 1 \end{bmatrix}$$



  - Unde w = $L_y - P_y$
  - **PROBLEME/LIMITARI:**
    - **limitarea la suprafețe plane**
    - datorită **problemelor de precizi**e este posibil ca **poligoanele proiectate sa fie afișate sub cele ale planului** pe care se proiectează umbrele
    - **umbrele sunt proiectate pe planul ce conține poligonul primitor** și nu direct pe acesta; astfel **umbra va fi desenată și în afara poligonului** -> **folosire stencil buffer pentru rezolvare**
    - **pot fi afișate umbre chiar și pentru obiectele care nu pot în mod normal să aibă umbre**; dacă obiectul este situat în spatele sursei de lumină umbrele generate de acesta se numesc **anti-umbre (anti-shadows)** iar dacă obiectul se află în spatele planului umbrele generate se numesc **umbre false (fake shadows**); o soluție pentru eliminarea acestor probleme ar fi **folosirea de clipping-planes**

Avantaje	**Dezvantaje**
- realizează proiectarea umbrelor pe suprafețe plane cu rezultate f. bune	- nu permite obiectelor concave sa se auto-umbrească
- metodă rapidă pentru scene mici cu o singură sursă de lumină și un plan pe care se proiectează umbrele	- creează hard shadows
	- dacă se dorește realizarea de umbre pentru surse de lumină multiple și mai multe plane de proiectare, costul computațional crește foarte repede

- **SHADOW VOLUMES**
  - se pot proiecta umbre arbitrare si se poate realiza auto-umbrirea obiectului
  - Pentru început sa ne imaginam **o sursă de lumină punctiformă și un poligon generator (shadower).**
  - Luând ca poligon generator un triunghi extinzând virtual liniile din sursa de lumină și care trec prin vârfurile triunghiului obținem o piramidă infinită.
  - Putem considera partea de deasupra ca fiind o piramidă iar partea de sub poligon ca fiind un trunchi de piramidă. **Orice parte a unui obiect care se află în interiorul volumului trunchiului de piramidă va fi în umbră**. Denumirea acestui volum este "volum de umbră" (shadow volume).



  - Să ne imaginăm că vizualizăm scena dintr-un anumit punct din spațiu iar pentru fiecare pixel afișat ducem o raza până când aceasta va lovi obiectul afișat pe ecran. Pe măsura ce raza este proiectată spre obiect incrementăm un contor de fiecare dată când aceasta intră în volumul de umbră (traversează o față front-facing a acestuia) și decrementăm același contor când raza iese din volumul de umbră (traversează o față back-facing a acestuia).
  - daca, atunci cand raza a lovit pixelul, contorul e > 0 atunci acel pixel se afla in volumul de umbra(e umbrit), in caz contrar se afla in afara acestuia
  - în practică se folosește stencil buffer-ul care va face contorizarea.
  - Folosind și stencil buffer-ul și z-buffer-ul se desenează pe rând întâi fețele front-facing ale volumului de umbră având grija să incrementăm contorul din stencil-buffer iar apoi sunt desenate fețele back-facing ale volumului de umbră având grijă să decrementăm contorul.
  - În final este afișată întreaga scenă, o valoare de 0 în stencil-buffer însemnând că acel pixel se află în afara volumului de umbră (este iluminat) iar o valoare de 1 înseamnă că acesta este umbrit.

Sinteză implementare algoritm:

1. Se dezactivează buffer-ele de culoare și adâncime referitor la scriere (acestea nu mai primesc update-uri) dar testul de adâncime rămâne activat
2. Se activează back-face culling-ul (vor fi desenate doar fețele front-facing)
3. Se setează operația stencil să incrementeze (GL_INCR) buffer-ul de marcaj dacă testul de adâncime reușește
4. Se desenează poligoanele volumului de umbră
5. Se activează front-face culling-ul (vor fi desenate doar fețele back-facing)
6. Se setează operația stencil să decrementeze (GL_DECR) buffer-ul de marcaj dacă testul de adâncime reușește
7. Se desenează poligoanele volumului de umbră

- Metoda denumită și depth-pass deoarece stencil buffer-ul este modificat când testul de adâncime reușește.
- Problemă când observatorul se află în interiorul unuia sau mai multe volume de umbră:
  - Se inițializează stencil buffer-ul cu numărul de volume de umbră în care se află observatorul;
  - Calculul dacă observatorul se află în cadrul mai multor volume de umbră poate deveni foarte costisitor -> folosire metoda depth-fail

Metoda depth-fail (stencil buffer-ul este modificat când testul de adâncime eșuează) :

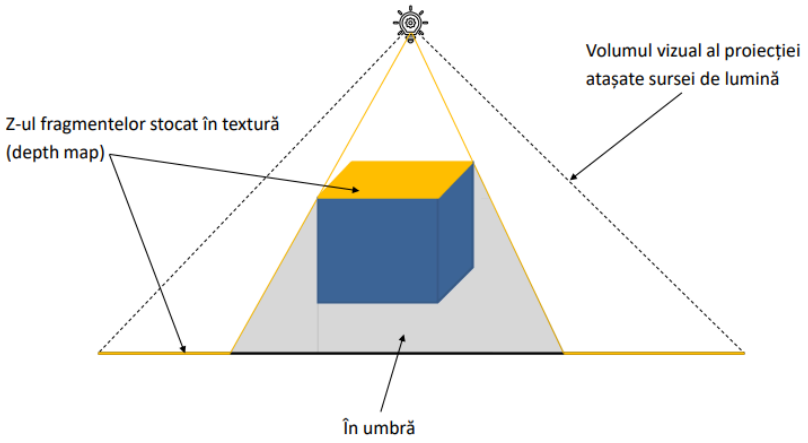
1. Se dezactivează buffer-ele de culoare și adâncime referitor la scriere (acestea nu mai primesc update-uri) dar testul de adâncime rămâne activat
2. Se activează front-face culling-ul (vor fi desenate doar fețele back-facing)
3. Se setează operația stencil să incrementeze (GL_INCR) buffer-ul de marcaj dacă testul de adâncime eșuează
4. Se desenează poligoanele volumului de umbră
5. Se activează back-face culling-ul (vor fi desenate doar fețele front-facing)
6. Se setează operația stencil să decrementeze (GL_DECR) buffer-ul de marcaj dacă testul de adâncime eșuează
7. Se desenează poligoanele volumului de umbră

Practic se numără poligoanele de umbra din spatele suprafețelor considerate !

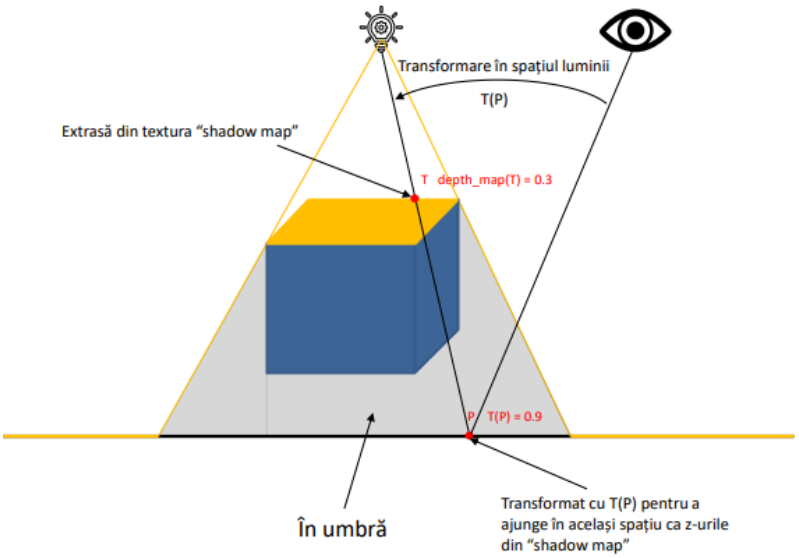
Observație: părțile din față și spate a volumului de umbră (bazele trunchiului piramidei) trebuie să fie construite (acoperite)

- **SHADOW MAPPING**
  - **2 PASI DE REDARE (RENDER PASS)**
    1. **Scena redată din poziția sursei de lumină cu redare direct într-o textură**

- matricea de VIEW calculata cu glm::lookat folosind ca poziție a observatorului poziția sursei de lumină
- Se setează ca **buffer de imagine un buffer diferit de cel al redării normale**; acest buffer **va avea atașată o textură imagine** de dimensiune dată -> denumită **shadow map / depth map**
- Se redă scena folosind matricea VIEW ce are ca poziție sursa de lumină **(practic scena este văzută din perspectiva sursei de lumină)** folosind ca buffer de imagine shadow map-ul
- Se vor memora în **shadow map doar coordonatele z ale fragmentelor vizibile** din poziția sursei de lumină:
  - Fragment shader-ul nu va scrie nimic în culoarea de output, rasterizatorul atașând automat coordonatele z interpolate în gl_FragDepth

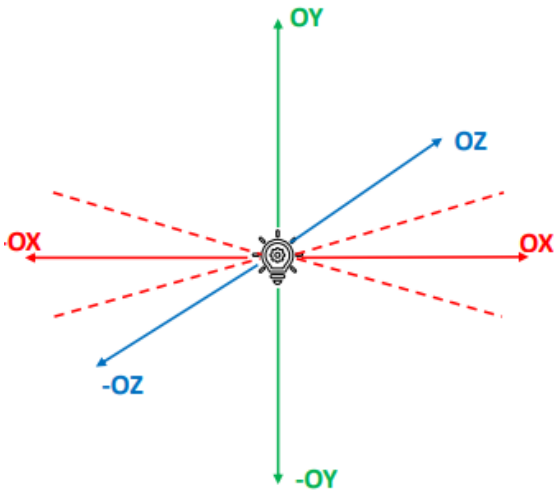


2. **scena redata normal**
- În vertex shader **se transformă vârfurile cu MVP**
- Suplimentar **se transmite matricea VP** pentru sursa de lumină ce va fi folosită **pentru a transforma vârfurile în spațiul sursei de lumină (MVPS)**
- În fragment shader **se extrage din textura de shadow map coordonata z corespondentă fragmentului considerat ca a fost transformat în spațiul sursei de lumină**
- Se compară aceasta valoare (extrasă din shadow map) cu cea a z-ului fragmentului considerat (transformat în spațiul sursei de lumină) și dacă **valoarea este mai mică** înseamnă ca atunci fragmentul considerat se află în **umbră (sursa de lumină "vede" alt fragment care este mai aproape de ea decât fragmentul considerat)**

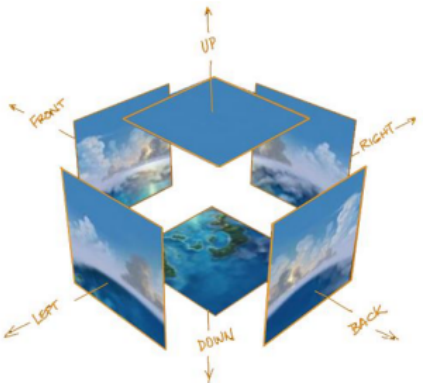


- **Omni-directional shadow-mapping**

- Sursa de lumină omni-direcționala (punctiformă)
- 6 volume de vizualizare din poziția sursei de lumină ce acoperă întreaga scenă: OX, -OX, OY, -OY, OZ, -OZ
- Se folosește un cubemap unde fiecare față reprezintă textura de shadow map pentru fiecare din cele 6 vizualizări
- Pentru a nu reda explicit de 6 ori se folosește un geometry shader care generează fiecare față a cubemap-ului, fiecare din aceste fețe cu matricea de VIEW corespunzătoare
- În rest algoritm similar cu cel de la shadow map clasic cu singura diferență că extragerea coordonatei Z din shadow map se face prin eșantionarea din cubemap pe direcția de la sursa de lumină la fragmentul considerat



<- cate o vedere din poz sursei de lumina care acopera fiecare din cele 6 directii



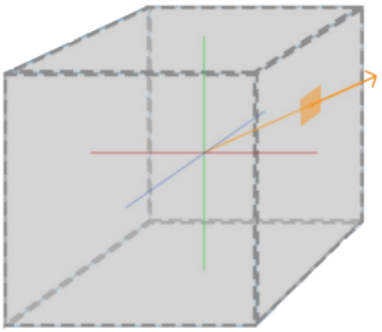
Cubemap

Eșantionare în cubemap folosind un vector direcți

Pentru omnidirectional shadow maps:

Eșantionare pe directia R

unde R = world_position - light_position



CURS 15

INTRODUCEREA UMBRELOR IN IMAGINI